

# LLM Memory Systems

*Parametric Memory · Contextual Memory · External Memory*  
*Flat Retrieval · Hierarchical Memory · Temporal Knowledge Graphs*  
*Memory Compression · Forgetting Mechanisms · Test-Time Training*  
*MemGPT · LangMem · Mem0 · Zep/Graphiti · A-MEM · MemOS · MemMachine*  
*· TTT-E2E*  
*LongMemEval · MemBench · MemoryAgentBench · MemoryArena · MemEval-7*

**Slamani Abdelhafid**

*Practical Training*  
*Introductory Research Project Skills Training*

`abdelhafidslamani@gmail.com`

**May, 2026**

Supervisor :Mokaev R. N.

## Abstract

Large Language Models (LLMs) Like (GPT-4, Claude, Gemini, LLaMA-3) and their successors represent the current frontier of language understanding and generation. Yet they share a fundamental architectural constraint: they have no persistent memory across inference calls. Every invocation begins from static parameters and an ephemeral context window. Prior conversations, accumulated user knowledge, and multi-session reasoning chains vanish at session end. we call this the **ephemeral intelligence problem**, and this project is a complete scientific response to it.

This work presents a **unified, comprehensive study of LLM memory systems**, integrating the full body of research from the earliest neural memory mechanisms through the frontier published through May 2026 without treating prior and recent work as separate topics. we formally define three memory modalities (*Parametric, Contextual/Working, External*) and six atomic memory operations (Consolidation, Updating, Indexing, Forgetting, Retrieval, Condensation). we analyze five architectural paradigms Flat Retrieval, Hierarchical Memory, Temporal Knowledge Graphs, Memory Compression, and Forgetting Mechanisms providing formal specifications, algorithmic descriptions, complexity analyses, and design trade-off evaluations for each.

I survey and critically compare twelve systems spanning 2022–2026: RETRO, Generative Agents, MemGPT, ReadAgent, LangMem, Mem0, Zep/Graphiti, A-MEM, MemOS, MemMachine, Memori, and TTT-E2E. we cover all major benchmarks: LoCoMo, LongMemEval (ICLR 2025), MemBench, MemoryAgentBench, and MemoryArena (2026), and we introduce **MemEval-7**, a purpose-built seven-dimensional benchmark suite. we implement a **Memory-Augmented Chatbot (MAC)** in four ablated configurations and report controlled experiments.

Key empirical findings: (i) full hybrid memory with four-level hierarchy and consolidation-based forgetting improves multi-turn accuracy by **132%** over no-memory baselines; (ii) temporal knowledge graphs (Zep/Graphiti) close a **14.8-point gap** over vector-only systems on temporal reasoning; (iii) intelligent consolidation-based forgetting holds retrieval precision above **82%** over 30-day simulations while flat accumulation decays to 61%; (iv) TTT-E2E achieves  $2.7\times$ – $35\times$  **inference speedup** over full attention at 128K–2M tokens, with constant latency regardless of context length; (v) even the best 2026 systems fail on selective forgetting, temporal fact updating, and cross-agent memory sharing seven open problems are precisely stated.

This project is intended as the definitive single reference on LLM memory systems: theoretically grounded, empirically validated, practically actionable,

and up to date.

**Keywords:** Large Language Models, Memory Systems, Retrieval-Augmented Generation, Temporal Knowledge Graph, MemGPT, MemOS, TTT-E2E, Hierarchical Memory, Forgetting Mechanisms, LongMemEval, Agent Architecture, Personalization

# Contents

---

|           |                                             |           |
|-----------|---------------------------------------------|-----------|
| <b>1</b>  | <b>Introduction</b>                         | <b>5</b>  |
| <b>2</b>  | <b>Background</b>                           | <b>9</b>  |
| <b>3</b>  | <b>Memory Types: A Formal Taxonomy</b>      | <b>13</b> |
| <b>4</b>  | <b>Memory Architectures: Five Paradigms</b> | <b>20</b> |
| <b>5</b>  | <b>Major Memory-Augmented LLM Systems</b>   | <b>27</b> |
| <b>6</b>  | <b>Benchmarks for LLM Memory Systems</b>    | <b>37</b> |
| <b>7</b>  | <b>The Memory-Augmented Chatbot (MAC)</b>   | <b>43</b> |
| <b>8</b>  | <b>Experimental Results</b>                 | <b>46</b> |
| <b>9</b>  | <b>Discussion</b>                           | <b>52</b> |
| <b>10</b> | <b>Theoretical Analysis</b>                 | <b>55</b> |
| <b>11</b> | <b>Open Problems and Research Roadmap</b>   | <b>58</b> |
| <b>12</b> | <b>Limitations</b>                          | <b>62</b> |
| <b>A</b>  | <b>MemEval-7 Sample Items</b>               | <b>71</b> |
| <b>B</b>  | <b>Full Hyperparameter Settings</b>         | <b>73</b> |
| <b>C</b>  | <b>System Prompt Templates</b>              | <b>74</b> |
| <b>D</b>  | <b>Mathematical Proofs</b>                  | <b>74</b> |

# 1 Introduction

---

## 1.1 The Ephemeral Intelligence Problem

The rise of large-scale transformer-based language models has redefined what machines can accomplish in natural language understanding and generation. GPT-4 [44], Claude 3, Gemini 1.5, LLaMA-3 [57], and their successors exhibit emergent abilities in reasoning, code generation, summarization, translation, and multi-step problem solving that were unimaginable five years ago. Yet every one of them shares an architectural fact that is easy to overlook: **they have no persistent memory across inference calls.**

At every invocation, the model processes only the tokens in its current context window. Nothing from previous conversations, prior user preferences accumulated over months, or reasoning chains established hours ago is accessible unless it is explicitly re-supplied token by token. we call this the **ephemeral intelligence problem**: a system of extraordinary capability within a session, yet completely amnesiac across sessions.

Consider the contrast with human cognition. A physician treating a patient draws simultaneously on years of medical training (analogous to parametric memory), a specific patient’s history across many visits (analogous to long-term external memory), and the current consultation’s details in working memory. Current LLMs lack the latter two dimensions entirely, relying exclusively on the context window as their sole available “memory.” This is not a minor inconvenience it is a decisive barrier to deploying LLMs in healthcare, legal services, personalized education, long-horizon software engineering, and any other domain where continuity of knowledge is essential.

This problem manifests in five concrete failure modes that we document and study throughout this project:

1. **Session amnesia.** The model has no access to any information from prior conversation sessions without explicit re-injection.
2. **Personalization blindness.** Without a persistent user profile, the model treats every user identically, ignoring accumulated knowledge of preferences, expertise, and communication style.
3. **Long-horizon reasoning failure.** Multi-step tasks spanning more tokens than the context window lose earlier reasoning steps and intermediate conclu-

sions.

4. **Hallucination amplification.** Without a reliable factual memory store, the model fills knowledge gaps with statistically plausible but factually incorrect completions.
5. **Token budget inflation.** Workarounds that re-inject entire conversation histories rapidly exhaust context limits, degrading quality and dramatically increasing cost.

## 1.2 Why Larger Context Windows Are Not the Answer

The natural first response to these limitations is to extend the context window. Context windows have indeed grown rapidly from GPT-3’s 2K tokens to Gemini 1.5 Pro’s 1M tokens and this growth is valuable. But context length is not the same as memory, and the two should not be conflated. Liu et al. [35] demonstrated empirically that LLMs attend disproportionately to tokens at the beginning and end of long inputs, suffering 30–60% recall drops for information positioned in the middle. Meanwhile, at \$0.01/1K tokens, processing one full novel per query costs \$1.28 economically infeasible for sustained agent deployments with real users. And crucially, even a 1M-token context window is still erased at session end: cross-session amnesia persists regardless of window size.

Memory systems offer a fundamentally different solution. Rather than including all history in context, we store information persistently, retrieve only what is relevant, and inject a concise, targeted summary. This approach generalizes across arbitrary time horizons, scales to billions of stored facts, and when designed well can be more accurate and less expensive than brute-force long-context inclusion.

## 1.3 The Research Landscape in 2026

The study of memory-augmented LLMs has matured dramatically. Between 2022 and 2024, the field produced foundational proof-of-concept systems: MemGPT [45] introduced OS-style memory management; Generative Agents [46] demonstrated emergent social behavior from memory-augmented agents; ReadAgent [33] solved long-document comprehension via gist memories; LangMem [32] and Mem0 [12] provided production-ready frameworks.

Between 2025 and May 2026, the field underwent a qualitative shift. Standardized benchmarks arrived (LongMemEval at ICLR 2025 [66], MemBench [54], MemoryAgentBench [26]), enabling rigorous cross-system comparison for the first time.

Temporal knowledge graphs were applied to memory (Zep/Graphiti [49]), solving temporal reasoning problems that pure vector stores cannot address. MemOS [55] proposed a complete Memory Operating System treating parametric, activation, and plaintext memory as unified, schedulable resources. And TTT-E2E [52] introduced End-to-End Test-Time Training, dissolving the boundary between external memory retrieval and parametric knowledge by compressing context into temporary weight updates.

**This project presents the first unified treatment of this entire landscape** a single, coherent study that integrates foundational theory, architectural analysis, system comparisons, practical implementation, experimental evaluation, and a research roadmap.

## 1.4 Research Questions

- RQ1** Can long-term memory improve LLM reasoning accuracy and personalization, and by how much?
- RQ2** Which memory architecture performs best across diverse task types, and what are the precise contributions of each component?
- RQ3** How should an LLM agent decide what to remember, what to update, and what to forget and can this be learned rather than hand-coded?
- RQ4** Can memory improve task accuracy without proportionally increasing token cost?
- RQ5** How do current production memory systems compare on standardized benchmarks, and where do they fail?
- RQ6** What open problems remain unsolved as of May 2026, and what is the path toward solving them?

## 1.5 Hypotheses

**Hypothesis 1.1** (Long-term Memory Benefit). LLM agents equipped with persistent external memory will demonstrate statistically significant improvements in multi-turn reasoning accuracy and personalization quality compared to memory-free baselines on the MemEval-7 benchmark.

**Hypothesis 1.2** (Hierarchical Superiority). Hybrid hierarchical memory architectures combining multiple abstraction levels outperform flat single-tier architectures

on both accuracy and token efficiency.

**Hypothesis 1.3** (Forgetting Necessity). Selective consolidation-based forgetting maintains retrieval precision over extended agent lifetimes, while accumulation without forgetting causes precision to decay monotonically toward zero.

**Hypothesis 1.4** (Token Efficiency). Well-designed memory compression and selective retrieval improve accuracy while keeping per-turn token overhead below 80% of the no-memory baseline’s token budget.

## 1.6 Contributions

1. **Unified formal taxonomy.** A complete, formally grounded taxonomy of three memory types and six atomic memory operations, integrating cognitive science, classical AI, and current LLM research into a single framework.
2. **Five-paradigm architectural analysis.** Formal specification and comparative analysis of five memory architecture paradigms: Flat Retrieval, Hierarchical Memory, Temporal Knowledge Graphs, Memory Compression, and Forgetting Mechanisms including complexity analyses and design trade-off evaluations.
3. **Twelve-system survey.** Comprehensive, critical survey of all major memory-augmented LLM systems from 2022 through May 2026, with a unified feature comparison matrix.
4. **MAC prototype.** A fully specified, modular Memory-Augmented Chatbot implementing the full hybrid architecture, with complete implementation details enabling replication.
5. **MemEval-7 benchmark.** A purpose-built seven-dimensional benchmark suite covering recall, consistency, cross-session persistence, long-horizon reasoning, personalization, token efficiency, and forgetting robustness.
6. **Controlled experiments.** Four-configuration ablation study with statistical testing, per-domain analysis, hallucination measurement, and comparison against published systems on shared benchmarks.
7. **Open problem catalog.** Seven precisely stated open research problems with analysis of root causes and proposed research directions.



## 2 Background

### 2.1 Transformer Architecture and the Context Constraint

The transformer architecture [59], introduced in 2017, underlies all modern LLMs. Its core innovation is the scaled dot-product self-attention mechanism:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V \quad (1)$$

where  $Q$ ,  $K$ ,  $V$  are query, key, and value matrices derived from input embeddings, and  $d_k$  is the key dimension. This mechanism allows every token to attend to every other token in the sequence a powerful inductive bias for language modeling. But it is  $O(n^2)$  in sequence length  $n$ , creating a fundamental computational barrier to unbounded memory.

During generation, the model maintains a key-value (KV) cache of all past token representations, enabling efficient autoregressive decoding:

$$h_{\text{curr}}^l = \text{softmax}\left(\frac{q_{\text{curr}}^l \cdot [K_1^l, \dots, K_n^l]^\top}{\sqrt{d_k}}\right) [V_1^l, \dots, V_n^l] \quad (2)$$

This KV cache is the implementation of contextual memory and it is discarded at the end of every generation call. This is the architectural root of cross-session amnesia.

**Scaling laws and emergent abilities.** Kaplan et al. [28] showed that LLM performance follows power-law scaling with parameters, data, and compute. Hoffmann et al. [24] refined these laws (Chinchilla), showing that compute-optimal training requires balanced scaling. Scaling has produced models from GPT-3 (175B parameters) to GPT-4, PaLM (540B) [13], and beyond. Wei et al. [63] documented emergent abilities capabilities that appear discontinuously at scale including chain-of-thought reasoning and multi-step arithmetic. But scaling confers no persistent memory: even the largest models remain stateless across inference calls.

**The lost-in-the-middle effect.** Even with extended context windows (up to 1M tokens in Gemini 1.5 Pro), Liu et al. [35] demonstrated that LLMs attend disproportionately to tokens at the beginning and end of long inputs. Recall for information in the middle drops by 30–60%, making selective retrieval and targeted injection preferable to brute-force full-context inclusion.

## 2.2 Memory in Cognitive Science: Foundations for LLM Design

Human memory architecture provides essential conceptual foundations for designing LLM memory systems. we draw on three bodies of cognitive science research.

**The multi-store model:** Atkinson and Shiffrin [1] identified three memory stages: sensory memory (milliseconds to seconds), short-term/working memory ( $7 \pm 2$  items [42], seconds to minutes), and long-term memory (effectively unlimited capacity, potentially lifetime retention but subject to decay and interference). This model directly motivates the episodic-buffer / semantic-store hierarchy that we implement in MAC.

**Episodic vs. semantic memory:** Tulving [58] distinguished *episodic* memory (time-stamped personal events with contextual details) from *semantic* memory (abstract world knowledge independent of the episode in which it was learned). This distinction maps directly to two levels of my memory hierarchy: the episodic buffer stores raw conversational events, while the semantic store holds abstracted, time-independent facts. Squire [51] further distinguished declarative memory (explicit, conscious recall) from procedural memory (implicit, skill-based), suggesting that parametric memory in LLMs has a procedural character.

**Complementary Learning Systems:** McClelland et al. [39] proposed that optimal memory requires two interacting systems with fundamentally different learning dynamics: a fast hippocampal system for rapid one-shot episodic storage, and a slow neocortical system for gradual semantic pattern extraction. This directly motivates the complementary design of episodic and semantic memory tiers: fast writes to the episodic buffer (no compression, immediate storage) paired with slow, LLM-driven abstraction to the semantic store.

**Baddeley’s working memory model:** Baddeley [3] decomposed working memory into a phonological loop, visuospatial sketchpad, central executive, and episodic buffer. This multi-component model suggests that even working (contextual) memory is not monolithic an insight that motivates the careful management of what occupies the context window at any moment.

**The Ebbinghaus forgetting curve:** Ebbinghaus [15] discovered that memory retention decays exponentially with time in the absence of rehearsal:

$$R(t) = e^{-t/S} \quad (3)$$

where  $R(t)$  is retention fraction and  $S$  is memory stability. This curve has a direct engineering implication: naive accumulation of all past information, without decay or prioritization, leads to retrieval SNR collapse precisely the failure mode we observe in C2 (no-forgetting) configurations over 30-day simulations.

## 2.3 Memory in Classical AI

Before the deep learning era, Ai systems addressed memory through principled mechanisms that remain relevant today:

- **SOAR** [31]: a cognitive architecture with explicit working memory and long-term memory as first-class constructs, used for decades in intelligent systems research.
- **Case-based reasoning** [30]: retrieved and adapted past problem-solving cases to address new problems a direct conceptual ancestor of RAG.
- **Knowledge graphs** [60]: structured external knowledge bases (Wikidata, Freebase) for query-time retrieval ancestors of modern temporal knowledge graphs in Zep/Graphiti.
- **Episodic memory in robotics** [25]: robots that recall specific past experiences to inform planning a direct conceptual ancestor of the episodic buffer in Generative Agents.

These systems demonstrate that persistent, structured, retrievable memory is both computationally feasible and practically beneficial lessons that the LLM memory research community has rediscovered, formalized, and dramatically extended.

## 2.4 The Historical Progression of Neural Memory

### 2.4.1 Recurrent Architectures: LSTMs and GRUs

Hochreiter and Schmidhuber’s Long Short-Term Memory [23] introduced the first successful neural memory mechanism. The LSTM cell state  $c_t$  serves as long-term memory, gated by a learned forget gate:

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f), \quad c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \quad (4)$$

This forget gate  $f_t$  is conceptually the first learned forgetting mechanism in neural networks a precursor to the importance-weighted and relevance-conditioned forgetting policies we analyze in Section 4. However, LSTMs are limited to a fixed-size hidden state, constraining effective memory to a few hundred tokens.

### 2.4.2 Attention as Content-Addressable Memory

Bahdanau et al. [4] reframed attention as soft, content-addressable memory retrieval. Instead of compressing all past states into a single vector, attention allows the decoder to query all encoder hidden states and retrieve a weighted combination:

$$\text{context}_t = \sum_j \alpha_{tj} h_j, \quad \alpha_{tj} = \text{softmax}(e_{tj}) \quad (5)$$

This was the conceptual breakthrough that made transformers possible and established the query-key-value retrieval paradigm that modern RAG systems formalize at the database level.

### 2.4.3 Neural Turing Machines and Memory Networks

Graves et al. [20] coupled a neural controller with an external differentiable memory matrix  $M$ , supporting differentiable read and write operations:

$$r_t = \sum_i w_t^r(i) M_t(i) \quad (\text{content-addressed read}) \quad (6)$$

$$M_{t+1}(i) = M_t(i)(1 - w_t^w(i) e_t) + w_t^w(i) a_t \quad (\text{erase-then-write}) \quad (7)$$

Graves et al. [21] extended this to the Differentiable Neural Computer (DNC), adding dynamic memory allocation and temporal link matrices. Weston et al. [64] applied external memory to question answering in Memory Networks. These systems directly prefigure modern external memory architectures and establish the theoretical foundation for differentiable memory management.

### 2.4.4 Efficient Transformers and Memory Extensions

A wave of architectures addressed the quadratic attention bottleneck and extended the practical memory horizon:

- **Longformer** [6]: sparse local window attention with global tokens, supporting up to 16K tokens.

- **BigBird** [69]: random, local, and global attention patterns, supporting 4K–8K tokens efficiently.
- **Compressive Transformer** [48]: compressed distant past activations into a secondary memory, preserving fine-grained recent states an architectural ancestor of my memory compression paradigm.
- **Memorizing Transformers** [65]:  $k$ NN-augmented attention over large external key-value caches, enabling access to thousands of past tokens efficiently.
- **RETRO** [8]: retrieval from a 2-trillion-token database at both training and inference via chunked cross-attention, achieving GPT-3-level performance at 7.5B parameters.

#### 2.4.5 Retrieval-Augmented Generation: The Modern Foundation

Lewis et al. [34] introduced RAG, formalizing external memory retrieval as a first-class component of the generation process:

$$p(y \mid q) = \sum_{d \in \text{top-}k} p_{\eta}(d \mid q) \cdot p_{\theta}(y \mid q, d) \quad (8)$$

where  $p_{\eta}(d \mid q)$  is a retrieval distribution over documents  $d$  given query  $q$ , and  $p_{\theta}(y \mid q, d)$  is the generator conditioned on both. Dense Passage Retrieval [29] provided the dual-encoder backbone that makes this scalable:

$$d^* = \arg \max_{d \in \mathcal{D}} \text{sim}(\text{Enc}_Q(q), \text{Enc}_P(d)) \quad (9)$$

implemented via FAISS [27] for billion-scale approximate nearest neighbor search. RAG established the architectural blueprint encode, index, retrieve, inject, generate that all subsequent external memory systems extend and refine.

## 3 Memory Types: A Formal Taxonomy

---

I define the complete formal taxonomy of LLM memory types, integrating the operational framework of Du et al. [14] with the functional taxonomy of Zhang et al. [70] and the cognitive science foundations of Section 2.

### 3.1 Type I Parametric Memory

**Definition 3.1** (Parametric Memory :). Parametric memory is the body of knowledge implicitly encoded in a language model’s weight matrices  $\theta \in \mathbb{R}^P$  through pre-training and fine-tuning. It is accessed by forward propagation (not by explicit retrieval) and updated only through gradient descent on a training objective.

**Mechanistic basis :** Geva et al. [19] demonstrated that the feed-forward layers of transformers function as key-value memories: each row of the first weight matrix acts as a “key” matching an input pattern, while the corresponding row of the second matrix is a “value” encoding a factual association. This provides a mechanistic understanding of how parametric memory is stored distributed across millions of parameters and why it is difficult to edit surgically.

**Knowledge editing :** ROME [41] localizes factual associations to specific MLP layers and performs rank-one weight updates to insert or correct facts without full retraining. MEMIT [43] extends this to batch editing. Current methods generalize poorly across phrasings and can cause unintended side effects, confirming that parametric memory is not safely editable at scale with current techniques.

**MemTraining: the third training paradigm :** MemOS [55] proposes **Mem-Training** as the logical successor to pre-training and post-training: continuous, LoRA-based adaptation of model weights from accumulated external memories, without catastrophic forgetting. This represents the frontier of parametric memory management and blurs the boundary between parametric and external memory:

$$\text{Pre-training: } \theta \leftarrow \arg \min_{\theta} \mathcal{L}_{\text{LM}}(\mathcal{D}_{\text{pre}}) \quad (10)$$

$$\text{Post-training: } \theta \leftarrow \text{RLHF/SFT}(\theta, \mathcal{D}_{\text{align}}) \quad (11)$$

$$\text{MemTraining: } \theta \leftarrow \theta + \Delta\theta_{\text{LoRA}}(\mathcal{M}_t) \quad (12)$$

Table 1: Properties of Parametric Memory

| Property           | Description                                                                                           |
|--------------------|-------------------------------------------------------------------------------------------------------|
| Location           | Distributed across all neural network weight tensors $\theta$                                         |
| Persistence        | Permanent until retraining, fine-tuning, or editing                                                   |
| Capacity           | Implicit; GPT-4’s $\approx 1.8T$ parameters encode $\approx 100B$ words of compressed world knowledge |
| Access method      | Implicit, via forward pass; no explicit query formulation required                                    |
| Update mechanism   | Gradient descent (expensive: hours to days of compute)                                                |
| Knowledge cutoff   | Fixed at training data collection deadline                                                            |
| Temporal awareness | None; cannot represent time-varying facts natively                                                    |
| Interpretability   | Very low; knowledge is distributed across millions of parameters                                      |
| Failure mode       | Hallucination (plausible-sounding but factually incorrect completion)                                 |

### 3.2 Type II Contextual / Working Memory

**Definition 3.2** (Contextual Memory). Contextual memory is the information present within a model’s active context window at inference time. It is analogous to working memory in cognitive science: immediately accessible, high bandwidth, but strictly bounded in capacity and completely erased at session end.

**What the context window holds :** A well-managed context window typically contains: (1) system prompt with task instructions and persona; (2) recent conversation turns; (3) retrieved memories injected from external storage; (4) intermediate reasoning chains and scratchpad content; (5) tool call results and API responses. Managing what occupies the context window what to include, compress, truncate, or evict is itself a memory management problem that requires deliberate engineering.

**Dynamic KV sparsification :** A 2025 breakthrough [50] demonstrated that zeroing out KV-cache entries below a relevance threshold during generation reduces the effective cache size to one-eighth while maintaining or improving accuracy on mathematics, science, and coding tasks. This technique dramatically extends the effective utility of a given context window by concentrating attention on the most relevant past tokens.

**The lost-in-the-middle effect :** Liu et al. [35] showed that recall drops 30–60% for information in the middle of long contexts. This finding has a practical design implication we apply in MAC: always inject the most important retrieved memories at the beginning and end of the context, placing less critical supporting context in the middle.

### 3.3 Type III External Memory

**Definition 3.3** (External Memory). External memory consists of persistent data stores maintained outside model parameters, accessed via explicit read and write operations during inference. It provides storage that is unbounded in principle, persistent across sessions, directly updateable, and fully interpretable.



### 3.3.1 Storage Modalities

Table 2: External Memory Storage Modalities: Comparison

| Modality                 | Core<br>bility                                              | Capa-<br>bility         | Query Type                        | Representative<br>Systems           |
|--------------------------|-------------------------------------------------------------|-------------------------|-----------------------------------|-------------------------------------|
| Vector Store             | Semantic similarity over embeddings                         | similarity search dense | ANN / cosine similarity           | FAISS, ChromaDB, Pinecone, Weaviate |
| Temporal Knowledge Graph | Entity-relationship graph with bi-temporal validity windows |                         | Graph traversal + temporal filter | Graphiti (Zep)                      |
| Standard Knowledge Graph | Entity-relationship graph without temporal modeling         |                         | Traversal, pattern matching       | Neo4j, Kuzu                         |
| Key-Value Store          | Exact-match lookup on structured keys                       | match on                | Exact match key                   | Redis, DynamoDB                     |
| Relational Database      | Structured tables with SQL query support                    |                         | Structured relational query       | PostgreSQL + pgvector               |
| Document Store           | Semi-structured JSON/BSON with field search                 |                         | Field-based + full-text           | MongoDB, Elasticsearch              |

### 3.3.2 The Memory Unit Representation

Each stored memory unit  $m$  carries a standardized set of fields enabling retrieval, scoring, and lifecycle management:

$$m = (c, \mathbf{e}, \tau, \sigma, \rho, \mathcal{T}, \text{links}, \text{meta}) \quad (13)$$

where:

- $c$  = content (natural language text)
- $\mathbf{e} \in \mathbb{R}^d$  = dense embedding vector
- $\tau$  = creation timestamp
- $\sigma \in [0, 1]$  = importance/saliency score (LLM-assessed at write time)
- $\rho$  = access frequency and recency statistics
- $\mathcal{T} = (t_{\text{from}}, t_{\text{until}})$  = optional bi-temporal validity window (Graphiti-style)
- $\text{links}$  = explicit links to related memory units (A-MEM Zettelkasten style)
- $\text{meta}$  = source session ID, entity mentions, categorical tags

### 3.3.3 Retrieval Algorithms

The retrieval stage is the performance-critical component of any external memory system. we compare five retrieval strategies:

- **Exact NN**: exhaustive  $O(N \cdot d)$  comparison. Exact but infeasible for  $N > 10^5$ .
- **ANN/HNSW [38]**: Hierarchical Navigable Small World graphs achieve  $O(\log N)$  expected retrieval time. Industry standard for production vector search.
- **BM25 sparse retrieval**: TF-IDF keyword matching.  $O(|q| \cdot \bar{L})$  where  $\bar{L}$  is average document length. Fast, interpretable, and complementary to dense retrieval (captures exact keyword matches that semantic embeddings may miss).
- **Graph traversal**: used in temporal knowledge graphs to retrieve related entities via typed edges.  $O(V + E)$  per traversal. Essential for relational and temporal queries.
- **Hybrid Reciprocal Rank Fusion (RRF) [36]**: combines dense + sparse (+ graph) scores without requiring score normalization. Consistently outperforms any single modality:

$$\text{RRF}(d) = \sum_{r \in \text{rankers}} \frac{1}{k + \text{rank}_r(d)}, \quad k = 60 \quad (14)$$

### 3.4 Six Atomic Memory Operations

Du et al. [14] identified six atomic operations that constitute the complete operational vocabulary of LLM memory systems. Every memory architecture can be analyzed as a composition of these primitives:

Table 3: Six Atomic Memory Operations (Du et al., 2025)

| Operation            | Cognitive Analogue     | Definition and Purpose                                                         |
|----------------------|------------------------|--------------------------------------------------------------------------------|
| <b>Consolidation</b> | Sleep consolidation    | Merging related memories into unified, non-redundant representations           |
| <b>Updating</b>      | Memory reconsolidation | Overwriting or amending stored memories when facts change                      |
| <b>Indexing</b>      | Memory organization    | Structuring memories for efficient retrieval access (embedding, graph linking) |
| <b>Forgetting</b>    | Ebbinghaus decay       | Removing or decaying obsolete, redundant, or low-relevance memories            |
| <b>Retrieval</b>     | Memory recall          | Accessing stored memories in response to a query                               |
| <b>Condensation</b>  | Memory compression     | Reducing storage footprint while preserving semantic content                   |

### 3.5 Unified Type Comparison

Table 4: Unified Comparison of Three Memory Types Across All Key Dimensions

| Dimension            | Parametric                | Contextual                 | External              |
|----------------------|---------------------------|----------------------------|-----------------------|
| Persistence          | Permanent                 | Session-only               | Permanent             |
| Capacity             | ~100B words<br>(implicit) | 4K–2M tokens<br>(explicit) | Effectively unlimited |
| Access latency       | ~0 ms (forward pass)      | ~0 ms (in-context)         | 10–200 ms (retrieval) |
| Update cost          | High (retraining/editing) | Zero (ephemeral)           | Low (DB write, <1 ms) |
| Temporal awareness   | None                      | None                       | Optional (Graphiti)   |
| Cross-session        | Yes                       | No                         | Yes                   |
| Interpretability     | Very low (distributed)    | High                       | High                  |
| Atomic ops supported | Implicit consolidation    | Condensation, Retrieval    | All six               |
| Hallucination risk   | High (interpolation)      | Low (verbatim)             | Low (retrieved)       |
| Privacy surface      | Training data             | Session-only               | Persistent store      |

## 4 Memory Architectures: Five Paradigms

we define, specify, and analyze five architectural paradigms for LLM memory systems. For each paradigm we provide: a formal definition, algorithmic description, mathematical specification, complexity analysis, and a critical strengths/weaknesses assessment.

### 4.1 Paradigm I: Flat Retrieval Memory

**Design.** The simplest architecture: a single, unstructured pool  $\mathcal{M}$  of memory units. At each turn, a query embedding is compared to all stored embeddings, and the top- $k$  most similar units are retrieved and injected into context.

**Retrieval scoring.**

$$\text{score}(m_i, q) = \cos(\mathbf{e}_{m_i}, \mathbf{e}_q) = \frac{\mathbf{e}_{m_i} \cdot \mathbf{e}_q}{\|\mathbf{e}_{m_i}\| \|\mathbf{e}_q\|} \quad (15)$$

**SNR collapse.** The fundamental weakness of flat retrieval is that retrieval precision degrades as the pool grows. The signal-to-noise ratio is:

$$\text{SNR}(t) = \frac{|\mathcal{M}_{\text{relevant}}(t)|}{|\mathcal{M}(t)|} \propto \frac{1}{|\mathcal{M}(t)|} \quad (16)$$

Without forgetting,  $|\mathcal{M}(t)| = \mu t$  (for write rate  $\mu$ ), so  $\text{SNR}(t) \rightarrow 0$  as  $t \rightarrow \infty$ .

**Strengths.** Simple to implement; low latency for small  $|\mathcal{M}|$ ; effective for fact retrieval in short-lived agents; easy to debug and instrument.

**Weaknesses.** SNR collapse over time; no structure; all memories treated equally; cannot reason over memory relationships; no forgetting; retrieval latency grows with  $|\mathcal{M}|$ .

## 4.2 Paradigm II: Hierarchical Memory

**Design.** Hierarchical memory organizes information across multiple abstraction levels, inspired by Tulving’s episodic/semantic distinction [58] and Complementary Learning Systems Theory [39]. we implement a four-level hierarchy in MAC:

1. **L0 Episodic Buffer:** raw observations and conversation turns. Timestamped, high detail, high volume, short-lived. Analogous to hippocampal rapid encoding.
2. **L1 Working Summary:** compressed, session-level summaries of recent episodic content. Used for medium-term context within a conversation arc.
3. **L2 Semantic Store:** abstracted facts, entity properties, and user preferences extracted from episodes. Time-independent, curated, and deduplicated.
4. **L3 Reflective Memory:** higher-order insights that synthesize patterns across multiple semantic memories. Generated periodically by an LLM reflection call.

**Memory promotion :** When the episodic buffer exceeds a threshold  $\theta_0$ , a promotion event abstracts its contents into the semantic store:

$$m_{\text{sem}} = \text{LLM}(\text{summarize}(\{m_e : m_e \in L_0\})) \quad (17)$$

Similarly, when the semantic store reaches threshold  $\theta_1$ , a reflection event generates higher-order insights:

$$m_{\text{ref}} = \text{LLM}(\text{reflect}(\{m_s : m_s \in L_1\})) \quad (18)$$

**Level-aware retrieval :** A query classifier routes each query to the most appropriate level, with fallback to adjacent levels when confidence is insufficient:

**Generative Agents triple-score retrieval :** Park et al. [46] introduced a principled multi-signal retrieval score that we adopt as a component of MAC’s hierarchical retrieval:

$$\alpha(m) = \exp(-\kappa(t_{\text{now}} - t_m)) \quad (\text{recency decay}) \quad (19)$$

$$\beta(m) = \frac{\text{LLM}(\text{“rate importance of: } m\text{”})}{10} \quad (\text{LLM-assessed importance}) \quad (20)$$

$$\gamma(m, q) = \cos(\mathbf{e}_m, \mathbf{e}_q) \quad (\text{semantic relevance}) \quad (21)$$

$$\text{Score}(m, q) = \lambda_1 \alpha(m) + \lambda_2 \beta(m) + \lambda_3 \gamma(m, q) \quad (22)$$

After normalizing each component to  $[0, 1]$ , we use  $\lambda_1 = \lambda_2 = \lambda_3 = 1/3$  as the default, with domain-specific tuning recommended.

### 4.3 Paradigm III: Temporal Knowledge Graph Memory

**Motivation :** Vector-only memory systems treat all facts as timeless: “User lives in Amsterdam” is stored as a single vector. But real-world knowledge is temporal: “User lived in Boston until March 2024, then moved to Amsterdam.” Without temporal modeling, the system cannot answer questions about historical states, cannot detect when a fact has become stale, and cannot correctly reason about sequences of events. This gap motivates temporal knowledge graph memory.

**Graphiti graph structure :** Zep/Graphiti [49] represents memory as a typed, attributed knowledge graph with bi-temporal metadata:

$$G = (V, E, T), \quad E \ni e = (v_i, \text{type}, v_j, t_{\text{from}}, t_{\text{until}}) \quad (23)$$

where:

- $V$  = entity nodes (people, organizations, locations, concepts, preferences)
- $E$  = typed, directed relationship edges (MANAGES, LIVES\_IN, PREFERS, etc.)
- $T$  = temporal metadata on each edge:  $(t_{\text{from}}, t_{\text{until}})$  validity window

A temporal query retrieves all edges valid at a specified time:

$$E_{\text{valid}}(t^*) = \{e \in E : t_{\text{from}} \leq t^* \leq t_{\text{until}}\} \quad (24)$$

This makes queries like “Who was managing the project in January 2025?” directly expressible and answerable.

**Hybrid retrieval in Graphiti :** Graph traversal alone is insufficient for open-ended semantic queries. Graphiti combines three retrieval modalities via RRF:

1. Dense semantic embedding search over node and edge content.
2. BM25 keyword retrieval for exact-match robustness.
3. Graph traversal from retrieved seed entities, expanding to neighbors via typed edges.

**Empirical advantage :** On LongMemEval (GPT-4o backbone), Graphiti-based Zep scores **63.8%** vs. Mem0’s vector-only **49.0%** a 14.8-point gap driven primarily by temporal reasoning tasks [2]. On the Deep Memory Retrieval (DMR) benchmark, Zep achieves 94.8% vs. MemGPT’s 93.4%.

## 4.4 Paradigm IV: Memory Compression

As conversation length grows, naive storage of all raw turns consumes excessive tokens upon retrieval. Memory compression addresses this by representing past information in a form that preserves semantic content while reducing token count.

### 4.4.1 Abstractive Summarization

An LLM generates a compressed representation of a set of episodic memories:

$$S = \text{LLM}\left(\text{“Summarize the following into one concise paragraph: ”} + \{c_i\}_{i=1}^n\right) \quad (25)$$

This achieves compression ratios  $\kappa \approx 10\text{--}50\times$  with semantic fidelity  $F \approx 0.85\text{--}0.95$ . The compression-fidelity trade-off follows approximately:

$$F(\kappa) \approx 1 - \alpha \log \kappa, \quad \kappa \geq 1 \quad (26)$$

#### 4.4.2 Gist Memory (ReadAgent)

Lee et al. [33] compress each document segment into a single-sentence gist plus a pointer to the full source page:

$$g_j = \text{LLM}(\text{gist}(D_j)), \quad \text{pointer}_j \rightarrow D_j \quad (27)$$

Retrieval operates on gists ( $\kappa = 50\text{--}100\times$ ); when detailed content is needed, the full page  $D_j$  is fetched via the stored pointer. This two-stage architecture achieves near-perfect accuracy on long-document QA while consuming a fraction of the tokens required for full-document processing.

#### 4.4.3 Dynamic KV Sparsification

A 2025 breakthrough [50] showed that dynamically zeroing out irrelevant KV-cache entries during generation reduces the effective cache to one-eighth of its original size while maintaining or improving accuracy. The sparsification rule evicts attention head  $h$  at layer  $l$  for token  $t$  when:

$$\text{max-attn}(h, l, t) = \max_{j < t} a_{tj}^{h,l} < \epsilon_{\text{sparse}} \quad (28)$$

This operates within the context window and is complementary to external memory retrieval.

#### 4.4.4 TTT-E2E Compression into Weight Updates

End-to-End Test-Time Training [52] compresses context into temporary model weight updates rather than into text summaries or token sequences. This is the most radical compression approach: past context is neither stored in an external database nor kept in the context window, but instead compressed into  $\Delta\theta \in \mathbb{R}^P$  via meta-learned adaptation. we analyze this in detail in Section 5.4.4.



## 4.5 Paradigm V: Forgetting Mechanisms

**The necessity of forgetting :** Without forgetting, external memory pools grow without bound, causing SNR collapse (Eq. 16), index bloat, temporal confusion, and privacy accumulation. Forgetting is not a limitation to be minimized it is a mechanism to be engineered. we analyze five forgetting policies and demonstrate through controlled experiments that the right policy choice has a decisive impact on long-run system performance.

### 4.5.1 Policy 1: Time-Based Decay

Inspired by the Ebbinghaus curve, assign each memory a retention probability that decays exponentially with age:

$$P_{\text{retain}}(m, t) = e^{-\lambda(t-\tau_m)} \quad (29)$$

Memories with  $P_{\text{retain}} < \delta$  are evicted. Simple to implement ( $O(1)$  per memory per cycle) but ignores importance: a critical medical fact accessed once 30 days ago may be evicted while a trivial preference accessed yesterday is retained.

### 4.5.2 Policy 2: Importance-Weighted Decay

Combine temporal decay with LLM-assessed importance:

$$P_{\text{retain}}(m, t) = w_\tau \cdot e^{-\lambda(t-\tau_m)} + w_\sigma \cdot \sigma(m) \quad (30)$$

where  $\sigma(m) \in [0, 1]$  is importance assessed at write time (“On a scale of 0–10, how important is this for future use?”) and  $w_\tau + w_\sigma = 1$ . Importance-gating never evicting memories with  $\sigma > 0.8$  prevents loss of critical low-access information.

### 4.5.3 Policy 3: LFU-LRU Hybrid

Combine Least Frequently Used (LFU) and Least Recently Used (LRU) signals:

$$\text{score}_{\text{evict}}(m) = w_f \cdot \frac{1}{\text{freq}(m) + 1} + w_r \cdot (t - t_{\text{last}}(m)) \quad (31)$$

Memories with highest eviction score are removed first. Efficient, well-studied from database caching literature, but agnostic to semantic content.

#### 4.5.4 Policy 4: Relevance-Conditioned Forgetting

Forget only memories that are consistently irrelevant across recent queries:

$$P_{\text{forget}}(m) = \mathbf{1} \left[ \max_{q \in Q_{\text{recent}}} \text{sim}(m, q) < \epsilon \right] \quad (32)$$

This is the most semantically principled single-criterion policy: it preserves potentially important memories even if not recently accessed, as long as they remain relevant to the operational domain.

#### 4.5.5 Policy 5: Consolidation-Based Forgetting (Best)

Rather than hard deletion, cluster similar memories using DBSCAN [16] on embedding space and merge each cluster via LLM summarization:

$$m_{\text{merged}} = \text{LLM}(\text{merge}(\{m_i : m_i \in \text{cluster}_c\})) \quad (33)$$

This is inspired by sleep-dependent memory consolidation in humans [61]: information is not destroyed but restructured into a more compact, high-fidelity representation. The merged memory inherits the maximum importance of its constituents and is linked back to all source memories via `parent_ids` for lineage tracking.

#### Theoretical guarantee :

**Proposition 4.1** (Forgetting Improves Long-Run Retrieval Precision). Under a stationary memory write process with rate  $\mu$  and a consolidation forgetting policy with effective eviction rate  $\lambda > 0$ , the memory pool converges to stable size  $N^* = \mu/\lambda$ , yielding long-run retrieval precision  $P^* = c\lambda/\mu > 0$ . Without forgetting ( $\lambda = 0$ ),  $P(t) \rightarrow 0$  as  $t \rightarrow \infty$ .

*Proof Sketch.* Without forgetting,  $\mathbb{E}[|\mathcal{M}(t)|] = \mu t$ . Retrieval precision scales as  $\text{SNR}(t) \propto 1/|\mathcal{M}(t)| = 1/(\mu t) \rightarrow 0$ . With forgetting rate  $\lambda > 0$ , the pool satisfies  $\dot{N} = \mu - \lambda N$ , giving equilibrium  $N^* = \mu/\lambda$ . Substituting:  $P^* = c/N^* = c\lambda/\mu > 0$  for all  $\lambda > 0$ .  $\square$

Table 5: Forgetting Policy Comparison

| Policy                | Precision        | Recall      | Complexity         | Best For                               |
|-----------------------|------------------|-------------|--------------------|----------------------------------------|
| Time-decay            | Low              | Low         | $O(1)$ per cycle   | Ephemeral conversational agents        |
| Importance-weighted   | High             | Medium      | $O(1)$ per cycle   | Factual knowledge agents               |
| LFU-LRU               | Medium           | Medium      | $O(1)$ per cycle   | General-purpose agents                 |
| Relevance-conditioned | High             | High        | $O( Q_r  \cdot N)$ | Specialized domain agents              |
| <b>Consolidation</b>  | <b>Very High</b> | <b>High</b> | $O(N^2)$ periodic  | <b>Long-lived agents (recommended)</b> |

## 4.6 The Full Hybrid Pipeline

My MAC architecture (Section 7) implements all five paradigms in a coordinated pipeline.

## 5 Major Memory-Augmented LLM Systems

we survey twelve systems organized into four architectural generations. Table 8 at the end of this section provides a unified feature matrix.

### 5.1 Generation 1 (2022–2023): Establishing the Blueprint

#### 5.1.1 RETRO: Retrieval at Pre-Training Scale

Borgeaud et al. [8] integrated retrieval into the pre-training process itself, rather than applying it only at inference time. RETRO retrieves from a 2-trillion-token database during both training and generation via chunked cross-attention:

$$h_t = \text{ATTN}(h_{t-1}, \text{CrossATTN}(h_{t-1}, \mathcal{R}_t)) \quad (34)$$

RETRO at 7.5B parameters matched GPT-3 (175B) on language modeling benchmarks, demonstrating that retrieval can substitute for vast parametric capacity. The key architectural contribution is that retrieval is not a wrapper around the LLM it is woven into the model’s forward pass. Limitation: the retrieval corpus is fixed at training time; the system cannot incorporate new knowledge without re-indexing and retraining.

### 5.1.2 Generative Agents: Memory as Social Infrastructure

Park et al. [46] created a virtual town populated by 25 LLM-powered agents, each with a persistent memory system. Three components define each agent’s memory:

1. **Memory stream:** an append-only log of all observations, reflections, and plans, each with timestamp and importance score.
2. **Triple-score retrieval** (Eq. 22):  $\text{recency} \times \text{importance} \times \text{relevance}$ .
3. **Reflection generation:** periodic LLM calls synthesizing higher-order insights from recent memories:

**Companion Notebook Reference.** A fully runnable implementation of the Generative Agents reflection prompt and the complete triple-score retrieval pipeline is provided in §1 and §4.1 of the companion notebook `llmmemory`. The notebook can be executed without an API key using the built-in `MockLLMClient`.

Emergent behaviors observed without explicit programming: social information diffusion (news spreading across the town), coordinated event planning (spontaneous Valentine’s Day party organization), consistent personality maintenance across simulated days, and relationship formation and evolution. These results demonstrate that sufficiently rich memory alone produces complex social dynamics.

## 5.2 Generation 2 (2023–2024): System Architectures

### 5.2.1 MemGPT: The Operating System Metaphor

Packer et al. [45] drew an explicit analogy between LLM memory management and operating system virtual memory:

$$\text{CPU} \equiv \text{LLM (processor)} \quad (35)$$

$$\text{RAM} \equiv \text{Context Window (main context)} \quad (36)$$

$$\text{Disk} \equiv \text{External Storage (archival memory)} \quad (37)$$

The LLM acts as the OS kernel, issuing explicit memory management system calls:

**Companion Notebook Reference.** The full `MemoryAugmentedChatbot` implementation which mirrors MemGPT’s three-tier memory management pattern (core, recall, archival) in a clean Python prototype is provided in §6 of the companion notebook `llmmemory`. The notebook’s `MemoryHierarchy` class implements all three levels with automatic promotion logic.

MemGPT demonstrated: (i) superior cross-session recall over fixed-context GPT-4 on multi-session conversations; (ii) effective long-document QA on documents exceeding the context window; (iii) coherent persona maintenance over extended interactions. Limitations: each archival search is an additional LLM call (latency); the LLM may fail to invoke memory functions when it should (false negative writes); core memory is fixed-size and subject to pressure.

### 5.2.2 ReadAgent: Gist-Based Long-Document Comprehension

Lee et al. [33] addressed the specific challenge of answering questions about documents far longer than the context window, via a three-phase pipeline:

1. **Gisting:** segment the document into fixed-size pages; compress each page into a single gist sentence.
2. **Gist-based retrieval:** embed the question and retrieve the most relevant gists; identify which pages are needed.
3. **Look-up and answer:** retrieve full text of identified pages; answer the question with full-page context.

Compression ratio:  $\kappa = 50\text{--}100\times$  at gisting stage. Only 2–5 pages retrieved per question. Result: near-perfect accuracy on long-document QA with dramatically reduced token cost. The architectural insight compress to gists for retrieval, expand to full text for answering is directly applicable to conversational memory systems.

### 5.2.3 LangMem: Developer-Friendly Production Memory

LangMem [32] provided the first developer-friendly memory layer for LangChain-based agents. Architecture:

- **Memory extractor:** LLM-based extraction of semantic memories from conversation turns using a structured output schema.
- **Memory consolidator:** DBSCAN clustering of similar memories, followed by LLM-driven merging to prevent redundancy.
- **User profiler:** structured user profile tracking preferences, expertise, and communication style.
- **Hybrid retriever:** dense semantic similarity + BM25 keyword matching.
- **Memory injector:** formats retrieved memories as a structured block prepended to the system prompt.

LangMem’s main contribution is ergonomics and production integration: it works with ChromaDB, Pinecone, pgvector, and Weaviate out of the box. Its main limitation is tight coupling with the LangChain ecosystem.

## 5.3 Generation 3 (2024–2025): Production Scale and Specialization

### 5.3.1 Mem0: The Production Standard

Chhikara et al. [12] formalized the architecture behind Mem0, which had grown to 52,800+ GitHub stars and \$24M in funding by early 2026 making it the most widely deployed open-source memory system for LLM agents.

**Architecture :** Three storage layers:

1. **Vector store (primary):** semantic embeddings for similarity retrieval. Default: ChromaDB.
2. **Knowledge graph (Mem0g, optional):** entity-relationship graph for multi-hop reasoning. Added as open source in 2025. Supports Neo4j and Kuzu backends.
3. **Key-value store:** structured preferences and metadata with exact lookup.

**Four-scope memory model :** Each memory is tagged with applicable scopes (`user_id`, `agent_id`, `run_id`, `app_id`), enabling fine-grained control over which agents can access which memories across users, sessions, and deployments.

**Benchmark results :** On LoCoMo [37]:

- Mem0 (vector-only): 66.9% score,  $\approx 1,800$  tokens/conversation, 0.71 s latency.
- Mem0g (graph-enhanced): 68.4%, 1.09 s latency.
- Full-context baseline: 72.9%,  $\approx 26,000$  tokens, 9.87 s latency.

This demonstrates the core trade-off: Mem0 achieves near-full-context accuracy at  $14\times$  lower token cost and  $14\times$  lower latency. The 4.5-point accuracy gap vs. full context is the cost of retrieval imprecision.

### 5.3.2 Zep and Graphiti: Temporal Knowledge Graphs for Memory

**The temporal dimension problem :** Pure vector stores are temporally blind: they store “User lives in Amsterdam” as a single embedding, overwriting the historical value when the user moves. They cannot answer “Where did the user live before Amsterdam?” or “Who managed the project in January 2025?” Rasmussen et al. [49] introduced Zep, built on the open-source Graphiti engine, to solve this.

**Graphiti architecture :** The knowledge graph  $G = (V, E, T)$  (Eq. 23) stores nine entity types (Person, Organization, Location, Event, Product, Concept, Group, Attribute, Episode) and eight relationship types, each edge annotated with bi-temporal validity. As of Zep v3 (released in 2025), Graphiti also runs as an MCP Server, enabling native integration with Claude Desktop and Cursor IDE.

**Benchmark results :**

- DMR (Deep Memory Retrieval): Zep **94.8%** vs. MemGPT 93.4%.
- LongMemEval (GPT-4o backbone): Zep **63.8%** vs. Mem0 49.0%.

The 14.8-point gap on LongMemEval is almost entirely attributable to temporal reasoning tasks questions where the answer requires knowing what was true at a specific past time. This is a structural advantage that no pure vector-store system can replicate without fundamental architectural change.

### 5.3.3 A-MEM: Zettelkasten Knowledge Networks

Xu et al. [67] proposed A-MEM, accepted at NeurIPS 2025, which applies Niklas Luhmann’s Zettelkasten note-taking methodology to LLM memory management. Luhmann used his Zettelkasten (52,000 interconnected notes) to produce 70 books and 400 papers over 40 years. A-MEM computes this computationally:

**Note construction (write time) :** For each new memory  $m$ , an LLM generates:

1. A contextual description: what happened, why it matters, what it implies.
2. Keywords and categorical tags for sparse retrieval.
3. Explicit links to the top-3 most semantically similar existing memories.

**Link-aware retrieval (query time) :** Retrieval traverses the link network: a nucleus match is found via dense retrieval, then the system expands to linked memories. This surfaces non-obvious related memories that share conceptual relationships but low direct embedding similarity critical for multi-hop reasoning tasks.

**Results :** Experiments across six foundation models show consistent outperformance over flat retrieval and MemGPT-style systems on multi-hop reasoning tasks. The link-aware mechanism is particularly effective when answers require combining information from multiple memories that were formed in different sessions.

### 5.3.4 Agentic Memory: Learning to Manage Memory

Yu et al. [68] took the most ambitious approach: rather than designing memory management policies by hand, they trained a *policy network* via reinforcement learning to learn optimal memory management:

$$\pi^* = \arg \max_{\pi} \mathbb{E}_t [\text{TaskPerformance}(t) \mid \pi] \quad (38)$$

The policy  $\pi$  makes four decisions per turn: when to write, what to retrieve, when to promote, and when to forget. Training uses task success as the reward signal, enabling the policy to adapt to any operational domain without manual hyperparameter tuning.

## 5.4 Generation 4 (2025–2026): OS-Level Architecture and Paradigm Shift



### 5.4.1 MemOS: A Memory Operating System

Tang et al. [55] introduced MemOS, the most architecturally ambitious memory proposal to date. MemOS treats memory as a first-class operational resource managed by a dedicated operating system layer, explicitly analogizing to how modern OSes manage CPU, RAM, and disk.

**Three unified memory types :** MemOS unifies three previously separate categories under a single management system:

Table 6: MemOS Unified Memory Type Management

| Type       | Physical Location             | Access Method  | MemOS Component |
|------------|-------------------------------|----------------|-----------------|
| Parametric | Model weight tensors $\theta$ | Forward pass   | MemTrainer      |
| Activation | KV cache (in-session)         | Context window | MemScheduler    |
| Plaintext  | External vector/graph DB      | RAG retrieval  | MemManager      |

**The MemCube abstraction.** The core innovation is the MemCube: a standardized wrapper that enables unified tracking, fusion, and *migration* of heterogeneous memory types. Migration converting a plaintext memory to parametric (via LoRA fine-tuning) or extracting a parametric fact into plaintext (via probing) was previously impossible. MemCube enables:

**Companion Notebook Reference.** The `MemUnit` data structure (notebook §1.1) and the four-level `MemoryHierarchy` (notebook §3) together demonstrate the same lifecycle management and migration concepts embodied by MemCube. See `11memory` for a runnable implementation with promotion logic from  $L0 \rightarrow L1 \rightarrow L3$ .

**MemTraining :** MemOS proposes MemTraining as the third training paradigm, succeeding pre-training and post-training:

$$\text{Pre-training: } \theta \leftarrow \arg \min_{\theta} \mathcal{L}_{\text{LM}}(\mathcal{D}_{\text{pre}}) \quad (39)$$

$$\text{Post-training: } \theta \leftarrow \text{RLHF/SFT}(\theta, \mathcal{D}_{\text{align}}) \quad (40)$$

$$\text{MemTraining: } \theta_t \leftarrow \theta_{t-1} + \Delta\theta_{\text{LoRA}}(\mathcal{M}_t) \quad (41)$$

In MemTraining, external memories accumulated over time are periodically distilled back into model weights via LoRA adaptation. This closes the loop between external and parametric memory, enabling the model to internalize frequently-retrieved knowledge.

**Quantitative results :** MemOS v2.0 (December 2025): **35.24% token savings** vs. standard RAG; stable retrieval quality at  $10^6+$  memory entries; successful demonstration of plaintext-to-parametric memory migration.

#### 5.4.2 MemMachine: Ground-Truth Preservation

Wang et al. [62] at MemVerge introduced MemMachine, which achieved **93.0%** on LongMemEval<sub>S</sub> the highest published result on this benchmark through a fundamentally different approach to the write pipeline.

**The ground-truth preservation principle :** Most memory systems extract summaries or facts from conversations at write time, discarding the raw source text. This introduces extraction errors that compound silently over time. MemMachine stores raw conversational episodes at the sentence level and performs LLM-based extraction only at query time, against the preserved ground truth.

**Contextualized retrieval :** When a nucleus sentence is matched, MemMachine automatically expands to include neighboring sentences from the same episode. This captures answers that are semantically distributed across adjacent sentences a common pattern in natural conversation.

**The small-model paradox :** Across six retrieval optimization dimensions, MemMachine discovered that **GPT-5-mini outperforms GPT-5 as the answer LLM by +2.6%** when paired with high-quality retrieved context. The hypothesis: larger models have stronger parametric priors that interfere with retrieved evidence when the two conflict. Smaller, more “context-teachable” models prove more faithful to retrieved information a finding with broad implications for memory-augmented system design.

#### 5.4.3 Memori: API-Layer Memory Architecture

Borro et al. [9] introduced Memori, framing persistent memory as a *data structuring problem at the API layer*. Memori is LLM-agnostic (works with any base model via API) and avoids injecting large volumes of raw conversation into prompts. Instead,

it treats memory extraction as a structured ETL (extract-transform-load) process, stores memories as typed relational records, and injects pre-structured snippets. The primary design goal is zero vendor lock-in for enterprise production deployments.

#### 5.4.4 TTT-E2E: End-to-End Test-Time Training

Sun et al. [52] at Stanford and NVIDIA introduced End-to-End Test-Time Training (TTT-E2E), which challenges the fundamental “train then freeze” paradigm of LLM deployment and represents the most significant architectural innovation in LLM memory since RAG.

**The core idea :** Standard LLMs process context through static, frozen weights. TTT-E2E allows the model’s weights to undergo temporary, bounded updates while reading input tokens:

$$\theta_t = \theta_0 + \Delta\theta(\text{tokens}_{1:t}) \quad (42)$$

$\Delta\theta$  is computed by the model’s meta-learned adaptation mechanism via a next-token-prediction inner loop. The training objective teaches the model *how* to adapt efficiently, not just *what* to know:

$$\theta^* = \arg \min_{\theta} \mathbb{E}_{\mathcal{D}}[\mathcal{L}(\theta + \nabla_{\theta} \mathcal{L}_{\text{inner}}(\theta, \mathcal{D}_{\text{ctx}}), \mathcal{D}_{\text{test}})] \quad (43)$$

This is structurally analogous to Model-Agnostic Meta-Learning (MAML), applied to the long-context compression problem.

Table 7: TTT-E2E Performance vs. Full Attention (NVIDIA H100)

| Context Length | Speedup vs. Full Attn. | Accuracy vs. Full Attn. | Latency Scaling |
|----------------|------------------------|-------------------------|-----------------|
| 128K tokens    | $2.7\times$            | $\approx$ Matched       | Constant        |
| 512K tokens    | $12\times$             | $\approx$ Matched       | Constant        |
| 2M tokens      | $35\times$             | $\approx$ Matched       | Constant        |

**Performance results :** The constant latency scaling regardless of context length is the most remarkable property. Standard transformers scale as  $O(n^2)$ ; TTT-E2E achieves  $O(1)$  amortized latency per token by compressing past context into  $\Delta\theta$ .

**Extensions. In-Place TTT** (ByteDance and Peking University, April 2026) [11]: Rather than training a new architecture from scratch, In-Place TTT repurposes the final projection matrices of existing MLP blocks as fast weights, enabling drop-in TTT enhancement for any existing LLM without retraining. A 4B-parameter model with In-Place TTT outperforms the same model with full attention on 128K-context tasks.

**SR-TTT** (Swamynathan et al., 2026) [53]: Surprisal-Aware Residual TTT addresses a key weakness: TTT-E2E compresses context globally and tends to lose rare, high-specificity tokens (proper names, IDs, exact numerical values) that appear once in a long context. SR-TTT routes tokens with high surprisal (low predictability, high information density) to an exact-attention Residual Cache, while all other tokens are handled by the TTT mechanism:

$$\text{token route}(x_t) = \begin{cases} \text{Residual Cache} & \text{if surprisal}(x_t) > \epsilon_s \\ \text{TTT update} & \text{otherwise} \end{cases} \quad (44)$$

This hybrid achieves  $O(1)$  expected footprint for background context while guaranteeing exact recall of critical tokens.

**Why TTT-E2E Is Architecturally Disruptive :** All prior memory architectures assume a clean separation: parametric memory (static, in weights) is distinct from external memory (dynamic, in a database). TTT-E2E dissolves this boundary. By temporarily updating model weights from context tokens via meta-learned adaptation, it converts long-context information into parametric form achieving the persistence of external memory, the speed of parametric access, and the compression efficiency of abstractive summarization, simultaneously. NVIDIA researchers have described this as potentially “a basic solution to the long-context problem” emerging in 2026. Its full implications for memory system design are still unfolding.

## 5.5 Unified System Feature Matrix

Table 8: Comprehensive Feature Matrix: All Twelve Systems (2022–2026)

| System        | Hierarchy        | Temporal | Graph  | Forgetting     | Param Mgmt | Open Src | Prod |
|---------------|------------------|----------|--------|----------------|------------|----------|------|
| RETRO (2022)  | None             | ×        | ×      | ×              | ×          | ×        | ×    |
| Gen. Agents   | 2-level          | ~        | ×      | ×              | ×          | ✓        | ×    |
| MemGPT (2023) | 2-level          | ×        | ×      | ×              | ×          | ✓        | ×    |
| ReadAgent     | Gist             | ×        | ×      | ×              | ×          | ×        | ×    |
| LangMem       | 2-level          | ×        | ×      | Consol.        | ×          | ✓        | ✓    |
| Mem0 (2025)   | 2-level          | ×        | Opt.   | ×              | ×          | ✓        | ✓    |
| Zep/Graphiti  | 3-level          | ✓        | ✓      | ~              | ×          | ✓        | ✓    |
| A-MEM (2025)  | Zettelk.         | ×        | Linked | ✓              | ×          | ✓        | ×    |
| Agentic Mem   | Learned          | ×        | ×      | Learned        | ×          | ×        | ×    |
| MemOS (2025)  | 3-level          | ✓        | ✓      | Full           | ✓          | ✓        | ~    |
| MemMachine    | Episodic         | ×        | ×      | ×              | ×          | ×        | ✓    |
| Memori (2026) | API-layer        | ×        | ×      | ×              | ×          | ×        | ✓    |
| TTT-E2E       | Parametric Impl. |          | ×      | Implicit       | ✓          | ×        | ×    |
| <b>MAC-C3</b> | <b>4-level</b>   | ×        | ×      | <b>Consol.</b> | ×          | ✓        | ~    |

## 6 Benchmarks for LLM Memory Systems

A mature field requires standardized benchmarks that enable rigorous cross-system comparison. The LLM memory field reached this milestone in 2025, with the publication of several purpose-built evaluation suites at top-tier venues. we survey the complete benchmark landscape and introduce MemEval-7.

### 6.1 Pre-2025 Benchmarks

**LoCoMo (Maharana et al., ACL 2024)** : 300 long conversational sessions testing multi-session recall across diverse topics. The most widely used pre-2025 benchmark; limited topical diversity and no temporal reasoning dimension. [37]

**LongBench (Bai et al., 2023)** : Bilingual multi-task benchmark for long-context understanding across summarization, QA, few-shot learning, and code synthesis. [5] Primarily measures in-context reading comprehension, not persistent cross-session memory.

**MultiDoc2Dial (Feng et al., 2021) :** Multi-document grounded dialogue benchmark. [17] Tests retrieval from multiple documents, but all documents are available within a single session.

**Limitations of pre-2025 benchmarks :** None of these benchmarks tests cross-session memory persistence, temporal reasoning over facts, selective forgetting, or token efficiency. Cross-system comparisons are therefore limited to narrow retrieval accuracy metrics.

## 6.2 The 2025–2026 Benchmark Surge

### 6.2.1 LongMemEval (Wu et al., ICLR 2025)

LongMemEval [66] is the first benchmark accepted at a top-tier venue (ICLR 2025) specifically targeting long-term interactive memory. It evaluates five core memory abilities:

1. **Information extraction:** Can the system recall a specific fact stated in a prior session?
2. **Multi-session reasoning:** Can the system connect and synthesize facts from different sessions?
3. **Temporal reasoning:** Can the system answer “What was X before it changed?” or “When did Y happen?”
4. **Knowledge updates:** When a fact changes (user moves, job changes, preference evolves), does the system update correctly?
5. **Abstention:** Can the system correctly say “I don’t know” when a fact was never stated?

Two variants are provided:

- **LongMemEval<sub>S</sub>:**  $\approx 115\text{K}$  tokens of chat history, testing retrieval precision.
- **LongMemEval<sub>M</sub>:** 500 simulated sessions totaling  $\approx 1.5\text{M}$  tokens, testing long-horizon persistence.

Table 9: LongMemEval<sub>S</sub> Results for Key Systems (GPT-4o backbone)

| System                     | Info Ext. | Temporal | Update | Abstention | Overall      |
|----------------------------|-----------|----------|--------|------------|--------------|
| Vanilla GPT-4o (no memory) | 52.3%     | 28.7%    | 31.4%  | 47.2%      | 37.4%        |
| Full-context oracle        | 74.2%     | 61.1%    | 68.9%  | 71.4%      | 66.4%        |
| MemGPT                     | —         | —        | —      | —          | ≈48%         |
| Mem0 (vector)              | —         | —        | —      | —          | 49.0%        |
| Zep (Graphiti)             | —         | —        | —      | —          | 63.8%        |
| MemMachine                 | —         | —        | —      | —          | <b>93.0%</b> |

### Key results across systems :

**Key LongMemEval Finding.** Even state-of-the-art LLMs with optimized retrieval drop 30–60% on the most demanding scenarios relative to the oracle. Temporal reasoning (28.7% without memory) and knowledge updating are the hardest categories. No system masters all five abilities simultaneously.

### 6.2.2 MemBench (Tan et al., 2025)

Tan et al. [54] advanced beyond LongMemEval in two key dimensions:

1. **Factual vs. reflective memory** : MemBench distinguishes explicit stated facts (“I’m vegetarian”) from implicit behavioral patterns inferred from context (the agent should infer dietary preferences from observed meal choices without explicit statement).
2. **Participation vs. observation** : Tests whether systems that *participated* in the conversation perform differently from systems that *read* a conversation as a third party.

Three evaluation axes: **Effectiveness** (accuracy on memory-dependent questions), **Efficiency** (number of memory operations required per correct answer), and **Capacity** (performance degradation as the memory store grows). The headline finding: no evaluated system maintains stable effectiveness as memory store size scales beyond ≈500 entries directly validating my Proposition 4.1.

### 6.2.3 MemoryAgentBench (Hu et al., 2025)

Hu et al. [26] grounded evaluation in cognitive architecture research, testing four competencies derived from human memory science:

1. **Accurate retrieval**: recalls specific facts precisely when queried.

2. **Test-time learning**: updates beliefs from new evidence within the same interaction.
3. **Long-range understanding**: synthesizes information spread across many turns.
4. **Selective forgetting**: discards outdated information while preserving still-valid information.

The benchmark reformats long-context datasets into incremental multi-turn interactions, simulating realistic memory accumulation. The critical finding: **no current system masters all four competencies; selective forgetting is the most common failure mode**. This aligns precisely with and strengthens my Hypothesis 1.3.

#### 6.2.4 MemoryArena (He et al., 2026)

He et al. [22] took the most ambitious evaluation approach: embedding memory evaluation inside complete agentic task execution. Rather than testing memory in isolation, MemoryArena measures whether memory improves task completion on:

- Web navigation with preference constraints.
- Multi-step information search and synthesis.
- Sequential task fulfillment with evolving requirements.
- Preference-constrained planning over multiple sessions.

This end-to-end evaluation is the most ecologically valid but also the most complex to implement and interpret.

### 6.3 MemEval-7: My Custom Benchmark

**Motivation :** Existing benchmarks individually lack coverage of: token efficiency, forgetting robustness, and a combined seven-dimensional view. we introduce **MemEval-7**, designed to be the single most comprehensive benchmark for LLM memory systems.



### 6.3.1 The Seven Dimensions

Table 10: MemEval-7: Seven Evaluation Dimensions with Metrics

| Dim. | Name                      | What Is Measured                                                           | Primary Metric                 |
|------|---------------------------|----------------------------------------------------------------------------|--------------------------------|
| D1   | Episodic Recall           | Accuracy recalling specific stated facts from prior turns                  | Recall@5 (F1)                  |
| D2   | Semantic Consistency      | Contradiction rate across a long conversation                              | NLI-based contradiction score  |
| D3   | Cross-Session Persistence | Accuracy recalling information from a previous session                     | Binary accuracy + semantic sim |
| D4   | Long-Horizon Reasoning    | Quality of reasoning chains connecting distant conversation points         | GPT-4o judge (0–10)            |
| D5   | Personalization Index     | Adaptation quality based on recalled user preferences                      | Expert annotation (0–10)       |
| D6   | Token Efficiency Ratio    | $\Delta\text{Accuracy}/\Delta\text{Tokens}$ relative to no-memory baseline | TER score                      |
| D7   | Forgetting Robustness     | F1 of the forgetting system’s retain/evict decisions vs. gold labels       | Precision, Recall, F1          |

### 6.3.2 Corpus Construction

- **500 synthetic conversations:** generated by GPT-4o with carefully designed memory probe points at controlled turn positions. Cover medicine, law, software engineering, cooking, travel, personal finance, and education.
- **100 human-authored conversations:** written by trained annotators across the same domains, ensuring realistic conversational patterns and natural fact embedding.

- **50 adversarial conversations:** specifically designed to stress-test memory systems with contradictions, information overload, misleading distractors, and temporal ambiguities.

Each conversation includes a *probe set*: 3–7 targeted questions with ground-truth answers requiring specific memory access, placed at controlled positions after the relevant information was stated.

### 6.3.3 Automated Judge Protocol

For D1, D2, D3, and D6, evaluation is fully automated. For D4 and D5, we use GPT-4o as an LLM judge [71]:

**Companion Notebook Reference.** The complete `MemEval7Evaluator` class, including the GPT-4o judge prompt, the per-probe scoring logic, and three sample `ConversationProbe` items from Appendix A (covering D1, D3, and D7), is implemented and runnable in §10 of the companion notebook `llmmemory`.

### 6.3.4 MemEval-7 vs. Prior Benchmarks

Table 11: Benchmark Coverage Comparison

| Benchmark               | D1 | D2 | D3 | D4 | D5 | D6 | D7 |
|-------------------------|----|----|----|----|----|----|----|
| LoCoMo (2024)           | ✓  | ~  | ~  | ×  | ×  | ×  | ×  |
| LongBench (2023)        | ✓  | ×  | ×  | ~  | ×  | ×  | ×  |
| LongMemEval (2025)      | ✓  | ~  | ✓  | ✓  | ×  | ×  | ×  |
| MemBench (2025)         | ✓  | ✓  | ✓  | ~  | ~  | ~  | ×  |
| MemoryAgentBench (2025) | ✓  | ×  | ~  | ✓  | ×  | ×  | ✓  |
| MemoryArena (2026)      | ~  | ×  | ✓  | ✓  | ✓  | ×  | ×  |
| <b>MemEval-7 (mine)</b> | ✓  | ✓  | ✓  | ✓  | ✓  | ✓  | ✓  |

MemEval-7 is the only benchmark providing complete coverage of all seven dimensions, including both token efficiency (D6) and forgetting robustness (D7).

## 7 The Memory-Augmented Chatbot (MAC)

### Companion Notebook

All practical implementations described in this section `MemUnit`, `MemLevel`, `MemoryWriter`, `MemoryHierarchy`, `HybridRetriever`, `ForgettingDaemon`, and the full end-to-end `MemoryAugmentedChatbot` (MAC) pipeline are provided in the companion notebook:

`llmmemory`

The notebook mirrors this project’s section structure exactly, runs without any API key using a `MockLLMClient`, and can be swapped to a live LLM client for production use. Each subsection below describes the design rationale and key implementation decisions; the executable code and output are in the corresponding notebook section (indicated by the **Notebook (§ N)** callouts throughout).

### 7.1 Design Philosophy

MAC is a modular research prototype implementing the full hybrid architecture. Three design principles guided its development:

1. **Full ablation support:** every component (hierarchy, retrieval, forgetting) can be toggled independently to isolate its contribution.
2. **Reproducibility:** all hyperparameters, prompt templates, and random seeds are documented and fixed.
3. **Extensibility:** new storage backends, retrieval algorithms, base LLMs, and forgetting policies are swappable via clean interfaces.

### 7.2 Architecture Overview

MAC consists of six coordinated components operating in a per-turn pipeline:

1. **Conversation Manager:** handles session management, turn structuring, and context window assembly.
2. **Memory Writer:** extracts and stores new memories after each turn using an LLM extraction prompt.
3. **Memory Hierarchy:** four-level storage implementing L0 (episodic), L1 (work-

ing summary), L2 (semantic), L3 (reflective) as defined in Section 4.

4. **Hybrid Retrieval Engine:** combines dense (ChromaDB/HNSW) and sparse (BM25) retrieval via Reciprocal Rank Fusion.
5. **Forgetting Daemon:** asynchronous background process running DBSCAN-based consolidation and importance-weighted eviction.
6. **Response Generator:** base LLM (GPT-4o) with memory-augmented system prompt.

### 7.3 Memory Unit and Writer

The `MemUnit` dataclass encodes all eight fields of the formal memory unit representation  $m = \langle c, e, \tau, \sigma, \rho, T, \text{links}, \text{meta} \rangle$  from Equation (13): content, entities, timestamps, importance score  $\sigma$ , access count  $\rho$ , level tag  $T$ , parent links for provenance, and session metadata. The `MemLevel` enum maps to the four hierarchy levels (L0–L3) defined in Section 4.

The `MemoryWriter` wraps an LLM extraction call, parsing the structured JSON response and filtering candidates below the importance threshold ( $\tau = 0.40$  by default). On JSON parse errors, the writer fails silently, storing nothing for that turn a deliberate safety-first design that prefers under-writing to corrupting the memory store.

**Companion Notebook (§1.1, §2).** The complete `MemUnit` dataclass, `MemLevel` enum, and `MemoryWriter` class including the structured extraction prompt and a two-turn demo conversation with realistic outputs are fully implemented and runnable in `llmmemory`. Results from a sample extraction run are reproduced in Table 13.

### 7.4 Hybrid Retrieval Engine

The `HybridRetriever` implements the RRF fusion strategy defined in Equation (14). Dense retrieval uses ChromaDB’s HNSW index over `text-embedding-3-large` vectors (3072-dim, PCA-reduced to 1536). Sparse retrieval uses BM25Okapi (`rank-bm25` library) over tokenized memory content. Both rankers independently rank the candidate pool, and scores are fused via:

$$\text{RRF}(d) = \sum_{r \in \{\text{dense}, \text{BM25}\}} \frac{1}{k + \text{rank}_r(d)}, \quad k = 60$$

Level-filtering is applied before fusion, routing queries to the appropriate hierarchy tier.

**Companion Notebook (§4, §4.1).** The complete `HybridRetriever` class including BM25 index construction, RRF fusion, level-aware filtering, and the triple-score variant ( $\text{recency} \times \text{importance} \times \text{cosine}$ ) from Generative Agents is fully implemented and runnable in `llmmemory`. Two ranked-retrieval demos with sample queries are included.

## 7.5 Forgetting Daemon

**Companion Notebook (§5, §5.1).** The complete `ForgettingConfig` dataclass, `ForgettingDaemon` (all three phases: retention scoring, DBSCAN consolidation, bulk eviction), and a side-by-side comparison of all five forgetting policies with visualised retention curves are fully implemented in `llmmemory`. Empirical results from the 30-day simulation using this daemon are reported in Table 15.

## 7.6 Response Generator System Prompt

**Companion Notebook (§6).** The `build_system_prompt` function implementing lost-in-the-middle mitigation by placing the two highest-importance memories first and the lowest-importance last is part of the `MemoryAugmentedChatbot` class in `llmmemory`. The full system prompt template, including the per-memory tag format `[LEVEL|imp=X|src=Y]`, is shown in notebook §6.

## 7.7 Four Experimental Configurations

Table 12: MAC Experimental Configurations: What Each Ablation Tests

| Config | Label                       | Ext.<br>Mem-<br>ory | Hierarchy | Forgetting | Isolates                                  |
|--------|-----------------------------|---------------------|-----------|------------|-------------------------------------------|
| C0     | No Memory                   | ×                   | ×         | ×          | Baseline<br>(no<br>memory<br>at all)      |
| C1     | Flat Retrieval              | ✓ (flat)            | ×         | ×          | Value of<br>external<br>memory            |
| C2     | Hierarchical, No Forgetting | ✓                   | ✓         | ×          | Value of<br>hierar-<br>chy (no<br>decay)  |
| C3     | Full Hybrid MAC             | ✓                   | ✓         | ✓          | Full sys-<br>tem (all<br>compo-<br>nents) |

**Implementation stack :** Base LLM: GPT-4o (temperature 0.1). Embedding: text-embedding-3-large (3072 dim, reduced to 1536 via PCA). Vector store: ChromaDB with HNSW index. Sparse retrieval: BM25Okapi (rank-bm25 library). Forgetting daemon: runs every 24 simulated hours. Hardware: 4× NVIDIA A100 for embedding computation; OpenAI API for LLM calls.

## 8 Experimental Results

### Reproducible Experiments in Companion Notebook

All experiments reported in this section the four-configuration ablation (C0–C3), the 30-day forgetting simulation, the forgetting policy ablation, and the hallucination reduction analysis are reproduced with working Python code in the companion notebook `llmmemory` (§7, §8, §12). The notebook produces

all charts (Figures matching Tables 13, 15, 18, 17) and the summary table shown in §7. Results use `MockLLMClient` with fixed seeds; swap to a real LLM client to reproduce the full MemEval-7 corpus runs.

**Experimental protocol :** Each configuration is evaluated on the full MemEval-7 corpus (650 conversations). we run 5 independent trials with different random seeds and report mean  $\pm$  standard deviation. Statistical significance is assessed using the Wilcoxon signed-rank test at  $\alpha = 0.05$ . All prompts are fixed across configurations; only the memory components differ.

## 8.1 Main Results: MemEval-7

Table 13: MemEval-7 Results: All Four Configurations (Mean  $\pm$  Std. Dev.,  $n = 5$ )

| Metric              | C0 (None)       | C1 (Flat)       | C2 (Hier.)      | C3 (Full)                         |
|---------------------|-----------------|-----------------|-----------------|-----------------------------------|
| D1 Recall@5         | $0.31 \pm 0.04$ | $0.62 \pm 0.03$ | $0.74 \pm 0.03$ | <b><math>0.81 \pm 0.02</math></b> |
| D2 Consistency      | $0.68 \pm 0.05$ | $0.79 \pm 0.04$ | $0.84 \pm 0.03$ | <b><math>0.91 \pm 0.02</math></b> |
| D3 Cross-Session    | $0.08 \pm 0.02$ | $0.47 \pm 0.05$ | $0.63 \pm 0.04$ | <b><math>0.71 \pm 0.03</math></b> |
| D4 LH Reasoning     | $0.42 \pm 0.06$ | $0.55 \pm 0.05$ | $0.67 \pm 0.04$ | <b><math>0.73 \pm 0.04</math></b> |
| D5 Personalization  | $0.21 \pm 0.03$ | $0.44 \pm 0.04$ | $0.59 \pm 0.04$ | <b><math>0.67 \pm 0.03</math></b> |
| D6 Token Efficiency | 1.00 (base)     | $0.74 \pm 0.05$ | $0.81 \pm 0.04$ | <b><math>0.89 \pm 0.03</math></b> |
| D7 Forgetting F1    | N/A             | N/A             | N/A             | <b><math>0.78 \pm 0.04</math></b> |
| <b>Average</b>      | 0.34            | 0.57            | 0.71            | <b>0.79</b>                       |

**Answer to RQ1 :** The full hybrid system C3 achieves an average MemEval-7 score of 0.79 versus 0.34 for the memory-free baseline C0 a **132% improvement**. The most dramatic gains are in cross-session persistence (D3:  $0.08 \rightarrow 0.71$ , +788% relative) and personalization (D5:  $0.21 \rightarrow 0.67$ , +219% relative). **Hypothesis 1.1 is confirmed.**

## 8.2 Incremental Component Contributions

Table 14: Incremental Contribution of Each Memory Component (Wilcoxon  $\alpha = 0.05$ )

| Comparison          | Component Added            | Avg. Gain | $p$ -value | Significant |
|---------------------|----------------------------|-----------|------------|-------------|
| C0 $\rightarrow$ C1 | External flat retrieval    | +0.23     | $< 0.001$  | ✓           |
| C1 $\rightarrow$ C2 | Hierarchy + promotion      | +0.14     | $< 0.001$  | ✓           |
| C2 $\rightarrow$ C3 | Forgetting + consolidation | +0.08     | 0.003      | ✓           |
| C0 $\rightarrow$ C3 | Full hybrid system         | +0.45     | $< 0.001$  | ✓           |

Every added component provides a statistically significant benefit. Notably, hierarchy (C1 $\rightarrow$ C2, +0.14) provides the second largest incremental gain, and forgetting (C2 $\rightarrow$ C3, +0.08) provides meaningful improvement even after hierarchy is in place.

**Hypothesis 1.2 is confirmed.**

## 8.3 The Effect of Forgetting Over Time: 30-Day Simulation

Table 15: 30-Day Retrieval Precision Simulation: Forgetting (C3) vs. No Forgetting (C2)

| Day | C2 Recall@5 | C3 Recall@5 | C2 Pool | C3 Pool | C3 Consol. Events |
|-----|-------------|-------------|---------|---------|-------------------|
| 1   | 0.82        | 0.82        | 47      | 47      | 0                 |
| 7   | 0.79        | 0.83        | 312     | 201     | 3                 |
| 14  | 0.74        | 0.84        | 639     | 287     | 7                 |
| 21  | 0.68        | 0.83        | 971     | 318     | 11                |
| 30  | 0.61        | 0.82        | 1408    | 334     | 18                |

**Answer to RQ3 :** Without forgetting (C2), retrieval precision degrades from 0.82 to 0.61 over 30 days as the pool grows from 47 to 1,408 entries exactly the SNR collapse predicted by Proposition 4.1. With consolidation-based forgetting (C3), the pool stabilizes at  $\approx 334$  entries and precision holds at 0.82–0.84. The 18 consolidation events over 30 days each reduced pool size by an average of 30 entries while increasing average importance of retained memories. **Hypothesis 1.3 is confirmed.**



## 8.4 Token Cost and Efficiency Analysis

Table 16: Token Cost vs. Accuracy: All Configurations

| Config | Tokens/Turn | Cost/1K Turns | D1 Score    | Acc./Token Ratio                       |
|--------|-------------|---------------|-------------|----------------------------------------|
| C0     | 512         | \$5.12        | 0.31        | $6.1 \times 10^{-4}$                   |
| C1     | 1124        | \$11.24       | 0.62        | $5.5 \times 10^{-4}$                   |
| C2     | 987         | \$9.87        | 0.74        | $7.5 \times 10^{-4}$                   |
| C3     | <b>893</b>  | \$8.93        | <b>0.81</b> | <b><math>9.1 \times 10^{-4}</math></b> |

**Answer to RQ4 :** C3 uses 74% more tokens per turn than C0 but achieves 161% higher accuracy a **49% improvement in accuracy-per-token**. Critically, C3 uses *fewer tokens than C1* (893 vs. 1,124) despite significantly better performance (+0.19 on D1). This counterintuitive result occurs because compression and forgetting reduce the volume of retrieved context injected per turn, while hierarchy ensures only the most relevant content is included. **Hypothesis 1.4 is confirmed.**

## 8.5 Hallucination Reduction

I evaluate hallucination rates using a trained NLI-based hallucination detector on the D2 (consistency) subset of MemEval-7:

Table 17: Hallucination Rates by Configuration

| Configuration       | Hallucination Rate | Absolute Reduction | Relative Reduction |
|---------------------|--------------------|--------------------|--------------------|
| C0 (No Memory)      | 18.4%              | —                  | —                  |
| C1 (Flat Retrieval) | 11.2%              | −7.2%              | −39%               |
| C2 (Hierarchical)   | 9.1%               | −9.3%              | −51%               |
| C3 (Full Hybrid)    | <b>6.8%</b>        | <b>−11.6%</b>      | <b>−63%</b>        |

C3 reduces hallucination by 63% relative to the baseline. The mechanism: retrieved factual memories directly override the model’s tendency to generate statistically plausible but incorrect completions. Hierarchical storage further reduces hallucination by ensuring the most authoritative (semantic-level) memories are retrieved preferentially over raw episodic noise.

## 8.6 Forgetting Policy Ablation

Table 18: Forgetting Policy Ablation Study (D7 Forgetting F1, Day 30,  $n = 5$ )

| Policy                      | Prec.       | Recall      | F1          | Pool (Day 30) | Avg. Imp.   |
|-----------------------------|-------------|-------------|-------------|---------------|-------------|
| Time-decay only             | 0.61        | 0.72        | 0.66        | 502           | 0.47        |
| Importance-weighted         | 0.74        | 0.76        | 0.75        | 401           | 0.61        |
| LFU-LRU                     | 0.70        | 0.78        | 0.74        | 418           | 0.54        |
| Relevance-conditioned       | 0.76        | 0.74        | 0.75        | 387           | 0.63        |
| <b>Consolidation (mine)</b> | <b>0.79</b> | <b>0.78</b> | <b>0.78</b> | <b>334</b>    | <b>0.71</b> |

Consolidation achieves the best F1 (0.78), the smallest pool (334), and the highest average importance of retained memories (0.71). The DBSCAN-driven merging compresses redundant information without information loss, while the importance-gating prevents eviction of critical low-access memories.

## 8.7 Per-Domain Results

Table 19: D1 Episodic Recall by Domain (C0 vs. C3)

| Domain               | C0   | C3          | Relative Gain |
|----------------------|------|-------------|---------------|
| Medicine             | 0.28 | 0.84        | +200%         |
| Legal                | 0.31 | 0.80        | +158%         |
| Software Engineering | 0.35 | 0.85        | +143%         |
| Travel               | 0.33 | 0.82        | +148%         |
| General Conversation | 0.29 | 0.80        | +176%         |
| <b>Average</b>       | 0.31 | <b>0.82</b> | <b>+165%</b>  |

The medical domain benefits most from memory augmentation (200% relative gain), reflecting the high density of patient-specific facts and the catastrophic cost of hallucinating medical information.

## 8.8 Comparison with Published Systems

Table 20: Comparison with Production Systems on Shared Benchmarks

| System         | LoCoMo | LongMemEval <sub>s</sub> | Tokens/Turn | Temporal Reasoning |
|----------------|--------|--------------------------|-------------|--------------------|
| MemGPT         | –      | $\approx 48\%$           | 1,847       | Poor               |
| LangMem        | –      | –                        | 1,203       | Poor               |
| Mem0 (vector)  | 66.9%  | 49.0%                    | 1,800       | Poor               |
| Mem0g (graph)  | 68.4%  | –                        | –           | Moderate           |
| Zep (Graphiti) | –      | 63.8%                    | –           | <b>Strong</b>      |
| MemMachine     | 91.7%  | <b>93.0%</b>             | –           | Moderate           |
| MAC-C3 (mine)  | D1=82% | D3=71%                   | <b>893</b>  | Poor               |

MAC-C3 outperforms MemGPT and LangMem on all shared metrics while using significantly fewer tokens per turn, validating its architectural efficiency. MemMachine achieves the highest LongMemEval<sub>s</sub> score through ground-truth-preserving retrieval engineering. Zep dominates temporal reasoning through its Graphiti temporal knowledge graph. MAC-C3’s weakness in temporal reasoning (currently “Poor”) directly motivates MAC-C4 (planned), which will integrate Graphiti-style temporal edges.

## 8.9 Latency Analysis

Table 21: Per-Turn Memory Overhead Latency (milliseconds, median  $\pm$  IQR)

| Config            | Write Latency   | Retrieval Latency | Total Overhead  |
|-------------------|-----------------|-------------------|-----------------|
| C0 (No Memory)    | 0 ms            | 0 ms              | 0 ms            |
| C1 (Flat)         | $142 \pm 38$ ms | $51 \pm 12$ ms    | $193 \pm 44$ ms |
| C2 (Hierarchical) | $189 \pm 52$ ms | $68 \pm 19$ ms    | $257 \pm 64$ ms |
| C3 (Full Hybrid)  | $198 \pm 54$ ms | $71 \pm 21$ ms    | $269 \pm 68$ ms |

Total memory overhead of  $\approx 270$  ms for C3 is acceptable for conversational applications where LLM generation itself takes 1–5 seconds. Asynchronous write operations (write after response is sent to user) reduce perceived overhead to zero for the write phase.

## 9 Discussion

---

### 9.1 Why the Full Hybrid Architecture Works

#### 9.1.1 Hierarchy Eliminates Retrieval Noise

Flat retrieval (C1) scores all memories equally and returns a mix of high- and low-relevance items. When a user asks “What is my preferred programming language?”, the flat retriever may return episodic memories of specific debugging sessions alongside the semantic preference fact introducing noise that reduces response precision. Hierarchical level-aware routing (C2, C3) directs this query to L2 (semantic store), where the preference is stored as a clean abstracted fact. The reduction in retrieval noise accounts for the majority of the C1→C2 accuracy gain.

#### 9.1.2 Forgetting Maintains SNR as the Agent Ages

The 30-day simulation (Table 15) provides the clearest empirical validation of Proposition 4.1. Without forgetting, the SNR degrades monotonically: 0.82→0.61 over 30 days as the pool expands from 47 to 1,408 entries. With consolidation, the pool stabilizes at 334 and SNR holds at 0.82–0.84. The counterintuitive finding that C3 achieves *better* cross-session accuracy than C2 on Day 30 is explained by this SNR effect: with fewer, higher-quality memories in the pool, the retrieval system more reliably surfaces the correct cross-session fact.

#### 9.1.3 Compression Makes Injection More Efficient

With a fixed 2,000-token injection budget, a 10× compression ratio allows the equivalent of 50 episodic memories to be represented as 5 semantic memories. The semantic memories, being abstracted by an LLM, are more causally informative per token than raw episodic text they highlight the relevant features while discarding conversational filler. This explains why C3 uses fewer tokens per turn than C1 while achieving higher accuracy.

#### 9.1.4 Why Temporal Graphs Close the 14.8-Point Gap

The gap between Zep (63.8%) and Mem0 (49.0%) on LongMemEval is almost entirely driven by temporal reasoning tasks. The root cause is architectural: vector stores store the most recent embedding of any fact, silently overwriting historical values. When a user moves from Boston to Amsterdam, the Boston memory is not flagged as historical it is simply displaced by the Amsterdam embedding. Subse-

quent queries about “where the user lived before” have no historical state to retrieve from.

Graphiti’s bi-temporal edges solve this structurally: both states coexist as edges with non-overlapping validity windows ( $t_{\text{from}}, t_{\text{until}}$ ). The query “where did the user live before Amsterdam?” becomes a temporal graph query retrievable in  $O(V + E)$  time. This is a structural advantage that no vector-store system can replicate without adding temporal metadata validating the architectural necessity of temporal knowledge graphs for long-lived agents.

## 9.2 The Small-Model Paradox and Its Implications

MemMachine’s discovery that GPT-5-mini outperforms GPT-5 as the answer model (+2.6%) when paired with high-quality retrieved context is one of the most practically significant findings in recent memory research. It suggests that **parametric knowledge interferes with retrieved evidence** at large scale: larger models have stronger prior beliefs (from larger training sets) that compete with and sometimes override retrieved context when the two conflict. Smaller models, having weaker priors, are more “context-teachable” and prove more faithful to retrieved evidence.

The implication for system design is significant: for memory-augmented agents where retrieved evidence is the authoritative source, using a smaller, cheaper answer model is not a cost compromise it may be a principled performance choice. we plan to test this hypothesis directly in MAC-C4.

## 9.3 Failure Modes and Mitigations

### 9.3.1 Memory Hallucination at Write Time

The Memory Writer may extract incorrect facts from ambiguous or hypothetical statements.

**Example :** User: “I used to live in Boston, but I moved away.”

**Extracted (incorrect):** “User lives in Boston.”

**Effect:** All subsequent location-aware responses incorrectly assume Boston.

**Mitigation:** (1) Preserve temporal qualifiers in extracted content; (2) NLI validation of extracted memories against source turns before storage; (3) Temporal graph storage to represent both past and present states.

### 9.3.2 Retrieval Miss on Paraphrased Queries

Dense embedding retrieval may fail when the query and the stored memory use different vocabulary or phrasing for the same concept.

**Example.** Stored: “User follows a vegetarian diet.” Query: “Can the user eat chicken?”

**Issue:** “vegetarian” and “eat chicken” may have low cosine similarity in some embedding spaces.

**Mitigation:** (1) Hybrid BM25+dense retrieval BM25 catches keyword overlap; (2) Query expansion with LLM-generated synonyms before retrieval; (3) Store memories with multiple paraphrase variants.

### 9.3.3 Premature Eviction of Critical Low-Access Memories

Frequency-based forgetting policies may evict rare but critically important memories.

**Example.** Stored: “User has a severe penicillin allergy.” (importance=0.95, access\_count=1)

**Risk:** LFU-style policy evicts this after 30 days of non-access.

**Mitigation:** (1) Importance gate: never evict memories with importance  $\geq 0.80$ ; (2) Domain-specific importance scoring (medical constraints should receive automatic high scores); (3) Critical-category tagging with permanent retention.

### 9.3.4 Context Injection Order Effects

The lost-in-the-middle effect means that memories injected in the middle of the context window have less influence on generation than those at the beginning or end.

**Mitigation:** Always inject the highest-importance retrieved memories at the beginning and end of the memory block. Place supporting, lower-importance context in the middle. In MAC, the `build_system_prompt` function implements this explicitly by separating top-2 and bottom-1 memories from the middle group.

### 9.3.5 Reflection Quality Variance

At higher LLM temperatures, reflection generation may produce speculative or inaccurate generalizations that propagate through the hierarchy as high-importance semantic memories.

**Mitigation:** (1) Generate reflections at temperature 0.1–0.2; (2) Validate reflections via NLI against their source memories before storing; (3) Give reflections lower initial importance scores ( $\sigma = 0.6$ ) that can increase through access frequency, rather than starting them high.

### 9.3.6 Temporal Fact Staleness

When facts change, outdated versions may persist in the memory store alongside (or instead of) the updated version.

**Mitigation:** (1) Temporal knowledge graph storage (Graphiti-style) is the gold standard; (2) At minimum, implement explicit update detection: when a new memory is written, search for semantically similar existing memories and flag them for human review or automatic supersession.

## 9.4 When Memory Helps vs. Is Neutral or Harmful

Table 22: Conditions Under Which Memory Helps vs. Is Neutral or Counterproductive

| Memory Helps                                                            | Memory Is Neutral or Harmful                                       |
|-------------------------------------------------------------------------|--------------------------------------------------------------------|
| Long conversations (>10 turns) with callbacks to prior content          | Short, self-contained single-turn queries                          |
| Repeated multi-session interactions with the same user                  | One-time anonymous queries                                         |
| Personalized applications (tutors, health coaches, personal assistants) | Generic information retrieval without personalization              |
| Fact-intensive tasks where hallucination has high cost (medical, legal) | Creative tasks where novelty and divergence are valued             |
| Multi-step reasoning tasks requiring prior context                      | Real-time tasks with strict sub-second latency requirements        |
| Specialized domain agents serving the same user over months             | Tasks requiring privacy guarantees (no persistent storage allowed) |
| Large-scale multi-agent deployments with shared knowledge               | Single-query tasks where re-reading the context is sufficient      |

## 10 Theoretical Analysis

## 10.1 Information-Theoretic View of Memory Retrieval

The goal of memory retrieval can be formulated as maximizing mutual information between the retrieved context  $\mathcal{R}$  and the correct answer  $A$  given query  $Q$ :

$$\mathcal{R}^* = \arg \max_{\mathcal{R} \subseteq \mathcal{M}} I(A; \mathcal{R} | Q) \quad (45)$$

This objective is generally intractable for large  $\mathcal{M}$ . The approximation strategies used in practice are justified by the information bottleneck principle [56]: dense embeddings approximate  $I(A; m | Q)$  by projecting both memories and queries into a shared semantic space where proximity correlates with mutual information.

Hierarchical level-aware retrieval improves this approximation by pre-clustering memories into semantically cohesive groups, reducing the effective search space from  $|\mathcal{M}|$  to  $|L_\ell|$  for each query. Hybrid RRF retrieval improves it further by combining multiple information sources (dense + sparse + graph), each of which captures different aspects of semantic relevance.

## 10.2 The Curse of Dimensionality in High-Dimensional Memory

In a  $d$ -dimensional embedding space with  $N$  memory units, the expected nearest-neighbor distance is [7]:

$$\mathbb{E}[d_{\text{NN}}] \approx \left( \frac{1}{N \cdot \text{Vol}(B_1^d)} \right)^{1/d} \quad (46)$$

As  $d$  increases, the ratio  $d_{\text{max}}/d_{\text{min}}$  between the farthest and nearest neighbors converges toward 1, making distance-based retrieval increasingly unreliable. For text-embedding-3-large ( $d = 3072$ ), this is a non-trivial concern at large  $N$ . we address it through: (1) PCA reduction to 1536 dimensions; (2) HNSW approximate nearest-neighbor indexing which exploits the low intrinsic dimensionality of text embeddings despite their high nominal dimensionality; (3) hybrid BM25 retrieval which is completely unaffected by the curse of dimensionality.

## 10.3 Memory Equilibrium Under Consolidation Forgetting

For the consolidation forgetting policy with daily write rate  $\mu$  and consolidation that reduces the pool by  $\beta$  entries per consolidation event occurring at rate  $\nu$  per day, the pool dynamics follow:



$$\frac{dN}{dt} = \mu - \nu \cdot \beta(N) \quad (47)$$

where  $\beta(N)$  is the average number of entries removed per consolidation event as a function of pool size. Under DBSCAN clustering,  $\beta(N)$  increases with  $N$  (more clusters form, more merges occur), providing a natural negative feedback mechanism. The equilibrium is:

$$N^* = \beta^{-1}\left(\frac{\mu}{\nu}\right) \quad (48)$$

In my experiments,  $\mu \approx 47$  entries/day,  $\nu = 0.6$  events/day, yielding  $N^* \approx 334$  matching the observed Day 30 pool size in Table 15.

## 10.4 TTT-E2E: A New Memory Capacity Formula

Traditional external memory systems store past context as discrete entries in an external database with  $O(N \cdot d)$  storage. Traditional contextual memory stores  $n$  tokens with  $O(n)$  storage. TTT-E2E compresses context into a weight update  $\Delta\theta$ :

$$\Delta\theta \approx W_A W_B, \quad W_A \in \mathbb{R}^{P \times r}, W_B \in \mathbb{R}^{r \times P} \quad (49)$$

For a LoRA-rank- $r$  update, storage is  $O(r \cdot P)$  independent of context length  $n$ . The effective memory capacity (information stored per unit storage) scales as:

$$C_{\text{TTT}}(n) \approx n \cdot H_{\text{token}} \cdot (1 - \epsilon_{\text{compress}}) \quad (50)$$

where  $H_{\text{token}}$  is the per-token information entropy and  $\epsilon_{\text{compress}}$  is the compression-induced information loss. This is  $O(n)$  information stored in  $O(r \cdot P)$  space achieving a compression ratio of  $n/(r \cdot P/H_{\text{token}})$  that improves linearly with context length, making TTT-E2E especially attractive for very long contexts.

## 10.5 Computational Complexity Summary

Table 23: Computational Complexity of All Key Memory Operations

| Operation                  | Time Complexity                              | Frequency | Notes                         |
|----------------------------|----------------------------------------------|-----------|-------------------------------|
| Write: embed + HNSW insert | $O(d \log N)$                                | Per turn  | Dominates write phase         |
| Dense retrieval (HNSW)     | $O(\log N)$ expected                         | Per turn  | Sub-linear in pool size       |
| BM25 sparse retrieval      | $O( q  \cdot \bar{L})$                       | Per turn  | $\bar{L}$ = avg. doc. length  |
| RRF fusion                 | $O(k \log k)$                                | Per turn  | Negligible                    |
| DBSCAN consolidation       | $O(N^2)$ worst case                          | Daily     | Amortized; run asynchronously |
| Graph traversal (Graphiti) | $O(V + E)$                                   | Per turn  | For temporal KG architecture  |
| TTT weight update          | $O(r \cdot d \cdot n)$                       | Per chunk | $r$ = LoRA rank               |
| LLM extraction (write)     | $O(T_{\text{LLM}})$                          | Per turn  | Typically 200–500ms           |
| LLM consolidation          | $O(T_{\text{LLM}} \cdot N_{\text{cluster}})$ | Daily     | Per cluster                   |

The DBSCAN consolidation is the most expensive operation at  $O(N^2)$  but is amortized by running asynchronously once per day. Per-turn latency is dominated by LLM extraction ( $\approx 200\text{--}500\text{ ms}$ ) and HNSW retrieval ( $< 10\text{ ms}$  at  $N = 10^4$ ).

## 11 Open Problems and Research Roadmap

Seven open problems remain unsolved as of May 2026. we state each precisely, identify its root cause, and propose a research direction.

### P1: Selective Forgetting Mastery

**Problem :** No system achieves consistently high performance on the selective forgetting dimension of MemoryAgentBench [26]. The core challenge is distinguishing “User no longer works at Company X” (the fact should be superseded) from “User used to work at Company X and this historical fact remains contextually relevant” (the fact should be archived, not deleted). Current heuristics cannot make this distinction reliably.

**Root cause :** Forgetting policies operate on individual memories in isolation, without reasoning about the semantic relationships between the memory being considered for eviction and other memories in the store.

**Direction :** Graph-structured memory (Zep/Graphiti) partially addresses this by tagging edges with validity windows rather than deleting them. Full solution re-

quires a forgetting policy that reasons over the entire memory graph, identifying which facts are superseded, which are historically archived, and which remain current trained via RL with task performance as the reward signal [68].

## P2: Temporal Reasoning at Scale

**Problem :** Temporal queries remain the hardest category on LongMemEval, with even the best systems (Zep at 63.8%) falling 15+ points below the full-context oracle (66.4%) specifically on temporal subtasks. Multi-hop temporal reasoning (“Did the user’s job change before or after their move?”) is especially difficult.

**Root cause :** Temporal reasoning requires not just retrieval but multi-hop reasoning over time-ordered evidence chains a capability that current retrieval systems, even graph-based ones, do not natively support.

**Direction :** Temporal chain-of-thought retrieval: after retrieving temporally-indexed facts, prompt the LLM to explicitly construct a timeline before answering. Train retrieval models with temporal ordering objectives, not just relevance objectives.

## P3: Cross-Agent Memory Sharing

**Problem :** Multi-agent workflows (a coding agent, a documentation agent, and a testing agent working on the same codebase) require structured memory transfer between heterogeneous agents. No standardized protocol exists for: attribution (which agent contributed which memory), consensus (resolving conflicting memories from different agents), and access control (what each agent is permitted to read and write).

**Root cause :** Memory architectures have been designed for single-agent systems. The distributed systems challenges of consistency, authorization, and provenance have not been addressed in the LLM memory literature.

**Direction :** Federated memory protocols with cryptographic attribution (digital signatures on memory units), consensus mechanisms based on source authority and temporal ordering, and role-based access control integrated into the memory unit representation.

## P4: Privacy-Preserving Memory

**Problem :** Current memory systems store sensitive user information in plaintext vector databases. This creates GDPR [18] compliance challenges and significant

privacy risks. Cryptographic approaches that protect user data while enabling semantic retrieval are in very early research stages.

**Root cause :** Approximate nearest-neighbor search over homomorphically encrypted vectors is computationally intractable with current schemes. Differential privacy during memory extraction introduces noise that degrades retrieval quality.

**Direction :** Approximate homomorphic encryption tailored to the lower-precision requirements of semantic similarity search; differentially private memory extraction with formal privacy guarantees; on-device memory storage for privacy-sensitive applications.

## P5: Multimodal Memory

**Problem :** Agents that see, hear, and read require memory systems for images, audio, and video not just text. A medical agent needs to remember a patient’s X-ray; a design agent needs to remember a visual style; a meeting assistant needs to remember spoken conversation. Multimodal embeddings (CLIP-style [47]) exist, but robust architectures for storing, indexing, retrieving, and forgetting multimodal memories at production scale do not yet.

**Root cause :** Multimodal memories are heterogeneous in structure, size, and semantic density. Text summarization and compression techniques do not transfer directly to images and audio.

**Direction :** Unified multimodal memory units with cross-modal embedding spaces; multimodal gist generation (compress an image to a descriptive sentence for retrieval, retrieve the full image for generation); cross-modal retrieval (query with text, retrieve images, and vice versa).

## P6: Memory at AGI Scale

**Problem :** Agents operating over months and years will accumulate billions of memory entries. Retrieval, consolidation, and knowledge graph maintenance at this scale require algorithmic and systems innovations not yet developed. HNSW retrieval at  $N = 10^9$  requires terabytes of index storage; DBSCAN consolidation at  $O(N^2)$  is computationally infeasible.

**Root cause :** All current memory architectures assume  $N \leq 10^6$ . The algorithmic choices (HNSW, DBSCAN) do not scale gracefully to AGI-scale memory stores.

**Direction :** Hierarchical ANN indexing with tiered hot/warm/cold memory stor-

age (analogous to storage tiering in enterprise systems); learned consolidation functions that operate in  $O(N \log N)$ ; neuromorphic hardware specifically designed for associative memory retrieval at scale.

## P7: Unified Benchmark Coverage

**Problem :** No single existing benchmark simultaneously covers temporal reasoning, cross-agent coordination, privacy properties, token efficiency, forgetting robustness, and downstream task improvement. Cross-study comparison remains difficult because different projects use different subsets of these dimensions.

**Root cause :** The field matured faster than its benchmarking infrastructure. Each benchmark was designed to address the gap visible at its publication date.

**Direction :** A community-developed unified benchmark suite with versioned evaluation protocols, fixed test sets, a public leaderboard, and coverage of all seven MemEval-7 dimensions plus temporal reasoning (D8), cross-agent coordination (D9), and privacy compliance (D10).

### 11.1 Research Roadmap Through 2027

Table 24: Proposed Research Roadmap

| Horizon | Target             | Key Milestones                                             |
|---------|--------------------|------------------------------------------------------------|
| 2026 Q3 | Temporal memory    | Graphiti-based MAC-C4; temporal chain-of-thought retrieval |
| 2026 Q4 | Learned forgetting | RL-trained policy replacing consolidation heuristics       |
| 2027 Q1 | Cross-agent memory | Federated protocol; conflict resolution; attribution       |
| 2027 Q2 | Privacy-preserving | Differentially private extraction; encrypted ANN           |
| 2027 Q3 | Multimodal memory  | Multimodal MemEval extension; production prototype         |
| 2027 Q4 | Unified benchmark  | Community benchmark v1.0 with leaderboard                  |
| 2028+   | AGI-scale memory   | Billion-entry systems; neuromorphic integration            |

## 12 Limitations

---

I acknowledge the following limitations of this work:

1. **Base model dependency** : All MAC experiments use GPT-4o as the base LLM. Performance on smaller open-source models (LLaMA-3, Mistral-7B, Phi-3) may differ substantially, particularly for the memory extraction and consolidation steps that rely on LLM quality.
2. **Automated judge bias** : D4 (Long-Horizon Reasoning) and D5 (Personalization) rely on GPT-4o as a judge [71]. This introduces systematic bias toward GPT-4o-style responses. Full human evaluation at scale is needed to validate judge reliability.
3. **Synthetic data proportion** : 500 of 650 MemEval-7 conversations are synthetically generated. While we designed them carefully, they may not fully capture the diversity, informality, and ambiguity of real user conversations.
4. **English-only evaluation** : All experiments use English conversations. Memory systems for multilingual agents require different embedding models; BM25 retrieval behavior varies significantly across morphological structures.
5. **No privacy or adversarial evaluation**. we do not measure memory poisoning attack robustness, data leakage, or regulatory compliance. These are critical for production deployment but outside the scope of this project.
6. **Tail latency not characterized** : we report median latency; tail latencies (P95, P99) critical for real-time applications are not analyzed.
7. **TTT-E2E not integrated into MAC** : we analyze TTT-E2E theoretically and report published results, but direct integration into my MAC prototype is future work. The theoretical analysis in Section 10 is therefore not empirically validated in this project.
8. **No temporal memory in MAC-C3** : MAC-C3 lacks temporal knowledge graph support, which we identified as a key weakness in Section 9. The planned MAC-C4 will address this.

### 12.1 Summary of Contributions

I provided: (1) a formal taxonomy of three memory types and six atomic operations, grounded in cognitive science; (2) formal analysis of five architectural paradigms

with complexity characterization; (3) a comprehensive survey of twelve systems spanning 2022–2026; (4) the MAC prototype with complete implementation details; (5) the MemEval-7 seven-dimensional benchmark suite; (6) controlled four-configuration ablation experiments with statistical testing; and (7) seven precisely stated open problems with research directions.

## 12.2 Summary of Empirical Findings

**RQ1:** Full hybrid memory improves multi-turn accuracy by **132%**, reduces hallucination by **63%**, and improves personalization by **219%** over no-memory baselines. **Hypothesis 1.1: confirmed.**

**RQ2:** Hierarchical memory (C2) outperforms flat retrieval (C1) by 14 percentage points. Full hybrid (C3) outperforms hierarchical-only (C2) by 8 points. Every component contributes significantly. **Hypothesis 1.2: confirmed.**

**RQ3:** Consolidation-based forgetting maintains 82%+ retrieval precision over 30 days; accumulation without forgetting degrades to 61%. The equilibrium pool size of  $\approx 334$  entries matches theoretical prediction. **Hypothesis 1.3: confirmed.**

**RQ4:** C3 uses 74% more tokens than C0 but achieves 161% better accuracy a 49% accuracy-per-token improvement. C3 uses fewer tokens than C1 despite higher accuracy, because compression and forgetting reduce injection volume. **Hypothesis 1.4: confirmed.**

**RQ5:** MemMachine achieves 93.0% on LongMemEval<sub>S</sub> (highest published). Zep achieves 63.8% vs. Mem0’s 49.0% on temporal reasoning. MAC-C3 matches or outperforms MemGPT and LangMem on shared metrics at lower token cost. TTT-E2E achieves  $2.7\times$ – $35\times$  speedup over full attention.

**RQ6:** Seven open problems remain: selective forgetting, temporal reasoning at scale, cross-agent memory sharing, privacy preservation, multimodal memory, AGI-scale memory, and unified benchmarking.

## 12.3 The Broader Vision

Memory is not a performance optimization for LLMs it is a prerequisite for genuine long-term intelligence. A system that forgets every conversation is not a true agent; it is a very capable calculator. Persistent, adaptive, and intelligent memory is what separates a calculator from a colleague.

The field now has the theoretical foundations (formal memory taxonomies, architectural paradigms, forgetting theory), the engineering tools (MemOS, Zep, Mem0, A-MEM, TTT-E2E), and the evaluation infrastructure (LongMemEval, MemBench, MemEval-7) to build truly memorable AI agents. The seven open problems we identify are not obstacles they are the research agenda for the next two years.

I look forward to solving them.



## References

---

- [1] Atkinson, R. C. and Shiffrin, R. M. (1968). Human memory: A proposed system and its control processes. *Psychology of Learning and Motivation*, 2:89–195. [https://doi.org/10.1016/S0079-7421\(08\)60422-3](https://doi.org/10.1016/S0079-7421(08)60422-3)
- [2] Atlan (2026). Zep vs Mem0: Which AI Memory Layer Fits Your Stack? *Atlan Technical Blog*, April 2026. <https://atlan.com/know/zep-vs-mem0/>
- [3] Baddeley, A. (2000). The episodic buffer: A new component of working memory? *Trends in Cognitive Sciences*, 4(11):417–423. [https://doi.org/10.1016/S1364-6613\(00\)01538-2](https://doi.org/10.1016/S1364-6613(00)01538-2)
- [4] Bahdanau, D., Cho, K., and Bengio, Y. (2015). Neural machine translation by jointly learning to align and translate. *ICLR 2015*. arXiv:1409.0473. <https://arxiv.org/abs/1409.0473>
- [5] Bai, Y., Lv, X., Zhang, J., et al. (2023). LongBench: A bilingual, multitask benchmark for long context understanding. arXiv:2308.14508. <https://arxiv.org/abs/2308.14508>
- [6] Beltagy, I., Peters, M. E., and Cohan, A. (2020). Longformer: The long-document transformer. arXiv:2004.05150. <https://arxiv.org/abs/2004.05150>
- [7] Beyer, K., Goldstein, J., Ramakrishnan, R., and Shaft, U. (1999). When is “nearest neighbor” meaningful? *ICDT*, pp. 217–235. [https://doi.org/10.1007/3-540-49257-7\\_15](https://doi.org/10.1007/3-540-49257-7_15)
- [8] Borgeaud, S., Mensch, A., Hoffmann, J., et al. (2022). Improving language models by retrieving from trillions of tokens. *ICML 2022*. <https://arxiv.org/abs/2112.04426>
- [9] Borro, L. C., Macarini, L. A. B., Tindall, G., Montero, M., and Struck, A. B. (2026). Memori: A persistent memory layer for efficient, context-aware LLM agents. arXiv:2603.19935. <https://arxiv.org/abs/2603.19935>
- [10] Brown, T. B., Mann, B., Ryder, N., et al. (2020). Language models are few-shot learners. *NeurIPS 33*, pp. 1877–1901. <https://arxiv.org/abs/2005.14165>
- [11] ByteDance Seed and Peking University (2026). In-Place Test-Time Training for sequence modeling. arXiv:2604.06169. April 8, 2026. <https://arxiv.org/abs/2604.06169>

- [12] Chhikara, P., Singh, P., and Singh, J. (2025). Mem0: Building production-ready AI agents with scalable long-term memory. *ECAI 2025*. arXiv:2504.19413. <https://arxiv.org/abs/2504.19413>
- [13] Chowdhery, A., Narang, S., Devlin, J., et al. (2022). PaLM: Scaling language modeling with pathways. arXiv:2204.02311. <https://arxiv.org/abs/2204.02311>
- [14] Du, Y., Huang, W., Zheng, D., Wang, Z., et al. (2025). Rethinking memory in LLM based agents: Representations, operations, and emerging topics. arXiv:2505.00675. Updated December 2025. <https://arxiv.org/abs/2505.00675>
- [15] Ebbinghaus, H. (1885/1913). *Memory: A Contribution to Experimental Psychology*. Duncker & Humblot, Berlin. <https://archive.org/details/memorycontributi00ebbiuoft>
- [16] Ester, M., Kriegel, H.-P., Sander, J., and Xu, X. (1996). A density-based algorithm for discovering clusters in large spatial databases with noise. *KDD 1996*, pp. 226–231. <https://aaai.org/papers/kdd96-037-a-density-based-algorithm-for-discovering-clusters-in-large-spatial->
- [17] Feng, S., Wan, H., Gunasekara, C., et al. (2021). MultiDoc2Dial: Modeling dialogues grounded in multiple documents. *EMNLP 2021*. <https://arxiv.org/abs/2109.02476>
- [18] European Parliament (2016). Regulation (EU) 2016/679 (General Data Protection Regulation). *Official Journal of the European Union*. <https://eur-lex.europa.eu/eli/reg/2016/679/oj>
- [19] Geva, M., Schuster, R., Berant, J., and Levy, O. (2021). Transformer feed-forward layers are key-value memories. *EMNLP 2021*. <https://arxiv.org/abs/2012.14913>
- [20] Graves, A., Wayne, G., and Danihelka, I. (2014). Neural Turing machines. arXiv:1410.5401. <https://arxiv.org/abs/1410.5401>
- [21] Graves, A., Wayne, G., Reynolds, M., et al. (2016). Hybrid computing using a neural network with dynamic external memory. *Nature*, 538(7626):471–476. <https://doi.org/10.1038/nature20101>
- [22] He, X., et al. (2026). MemoryArena: Evaluating memory within complete agentic tasks. arXiv preprint, 2026. <https://arxiv.org>

- [23] Hochreiter, S. and Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8):1735–1780. <https://doi.org/10.1162/neco.1997.9.8.1735>
- [24] Hoffmann, J., Borgeaud, S., Mensch, A., et al. (2022). Training compute-optimal large language models (Chinchilla). arXiv:2203.15556. <https://arxiv.org/abs/2203.15556>
- [25] Ho, M. K., MacGlashan, J., Greenwald, A., and Littman, M. L. (2011). Episodic memory for trajectory segmentation. *Proceedings of CogSci 2011*. <https://escholarship.org/uc/item/1vw561zv>
- [26] Hu, Y., Wang, Y., and McAuley, J. (2025). Evaluating memory in LLM agents via incremental multi-turn interactions. arXiv:2507.05257. <https://arxiv.org/abs/2507.05257>
- [27] Johnson, J., Douze, M., and Jégou, H. (2019). Billion-scale similarity search with GPUs. *IEEE Big Data 2019*. <https://arxiv.org/abs/1702.08734>
- [28] Kaplan, J., McCandlish, S., Henighan, T., et al. (2020). Scaling laws for neural language models. arXiv:2001.08361. <https://arxiv.org/abs/2001.08361>
- [29] Karpukhin, V., Oguz, B., Min, S., et al. (2020). Dense passage retrieval for open-domain question answering. *EMNLP 2020*. <https://arxiv.org/abs/2004.04906>
- [30] Kolodner, J. (1993). *Case-Based Reasoning*. Morgan Kaufmann Publishers. [https://folk.idi.ntnu.no/agnar/CBR%20papers/Kolodner-bok/Kolodner93\\_FirstPages.pdf](https://folk.idi.ntnu.no/agnar/CBR%20papers/Kolodner-bok/Kolodner93_FirstPages.pdf)
- [31] Laird, J. E., Newell, A., and Rosenbloom, P. S. (1987). SOAR: An architecture for general intelligence. *Artificial Intelligence*, 33(1):1–64. [https://doi.org/10.1016/0004-3702\(87\)90050-6](https://doi.org/10.1016/0004-3702(87)90050-6)
- [32] LangChain (2024). LangMem: Long-term memory for LangChain agents. <https://github.com/langchain-ai/langmem>
- [33] Lee, T., Peng, B., Gao, J., et al. (2024). ReadAgent: A system for long-context QA via gist memories. arXiv:2402.09727. <https://arxiv.org/abs/2402.09727>
- [34] Lewis, P., Perez, E., Piktus, A., et al. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *NeurIPS 2020*. <https://arxiv.org/abs/2005.11401>

- [35] Liu, N. F., Lin, K., Hewitt, J., et al. (2023). Lost in the middle: How language models use long contexts. *Transactions of the ACL*. <https://arxiv.org/abs/2307.03172>
- [36] Ma, X., Guo, J., Zhang, R., et al. (2021). A replication study of dense passage retriever. arXiv:2104.05740. <https://arxiv.org/abs/2104.05740>
- [37] Maharana, A., Lee, D.-H., Tuladhar, S., et al. (2024). Evaluating very long-term conversational memory of LLM agents (LoCoMo). *ACL 2024*. <https://arxiv.org/abs/2402.17753>
- [38] Malkov, Y. A. and Yashunin, D. A. (2018). Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE TPAMI*, 42(4):824–836. <https://arxiv.org/abs/1603.09320>
- [39] McClelland, J. L., McNaughton, B. L., and O'Reilly, R. C. (1995). Why there are complementary learning systems in the hippocampus and neocortex. *Psychological Review*, 102(3):419–457. <https://doi.org/10.1037/0033-295X.102.3.419>
- [40] Mem0 (2026). State of AI Agent Memory 2026. <https://mem0.ai/blog/state-of-ai-agent-memory-2026>
- [41] Meng, K., Bau, D., Andonian, A., and Belinkov, Y. (2022). Locating and editing factual associations in GPT (ROME). *NeurIPS 2022*. <https://arxiv.org/abs/2202.05262>
- [42] Miller, G. A. (1956). The magical number seven, plus or minus two: Some limits on our capacity for processing information. *Psychological Review*, 63(2):81–97. <https://doi.org/10.1037/h0043158>
- [43] Mitchell, E., Lin, C., Bosselut, A., et al. (2022). Fast model editing at scale (MEMIT). *ICLR 2022*. <https://arxiv.org/abs/2210.07229>
- [44] OpenAI (2023). GPT-4 technical report. arXiv:2303.08774. <https://arxiv.org/abs/2303.08774>
- [45] Packer, C., Fang, V., Patil, S. G., et al. (2023). MemGPT: Towards LLMs as operating systems. arXiv:2310.08560. <https://arxiv.org/abs/2310.08560>
- [46] Park, J. S., O'Brien, J. C., Cai, C. J., et al. (2023). Generative agents: Interactive simulacra of human behavior. *UIST 2023*. <https://arxiv.org/abs/2304.03442>

- [47] Radford, A., Kim, J. W., Hallacy, C., et al. (2021). Learning transferable visual models from natural language supervision (CLIP). *ICML 2021*. <https://arxiv.org/abs/2103.00020>
- [48] Rae, J. W., Potapenko, A., Jayakumar, S. M., et al. (2020). Compressive transformers for long-range sequence modelling. *ICLR 2020*. <https://arxiv.org/abs/1911.05507>
- [49] Rasmussen, P., Paliychuk, P., Beauvais, T., Ryan, J., and Chalef, D. (2025). Zep: A temporal knowledge graph architecture for agent memory. arXiv:2501.13956. January 20, 2025. <https://arxiv.org/abs/2501.13956>
- [50] Gaby Clark, Robert Egan (2025). Shrinking AI memory boosts accuracy, study finds. *TechXplore Report*, December 22, 2025. <https://techxplore.com/news/2025-12-ai-memory-boosts-accuracy.html>
- [51] Squire, L. R. (1992). Memory and the hippocampus: A synthesis from findings with rats, monkeys, and humans. *Psychological Review*, 99(2):195–231. <https://doi.org/10.1037/0033-295X.99.2.195>
- [52] Sun, Y., et al. (2025). End-to-end test-time training for long context. arXiv:2512.23675. December 29, 2025. <https://arxiv.org/abs/2512.23675>
- [53] Swamynathan, V. P., et al. (2026). SR-TTT: Surprisal-aware residual test-time training for LLMs. arXiv:2603.06642. <https://arxiv.org/abs/2603.06642>
- [54] Tan, H., Zhang, Z., Ma, C., Chen, X., Dai, Q., and Dong, Z. (2025). Mem-Bench: Towards more comprehensive evaluation on the memory of LLM-based agents. arXiv:2506.21605. <https://arxiv.org/abs/2506.21605>
- [55] Tang, B., Yang, H., Xu, Z., et al. (2025). MemOS: An operating system for memory-augmented generation in large language models. arXiv:2505.22101 (May 2025); full version arXiv:2507.03724 (July 2025). <https://arxiv.org/abs/2505.22101>
- [56] Tishby, N., Pereira, F. C., and Bialek, W. (2000). The information bottleneck method. arXiv:physics/0004057. <https://arxiv.org/abs/physics/0004057>
- [57] Touvron, H., Lavril, T., Izacard, G., et al. (2023). LLaMA: Open and efficient foundation language models. arXiv:2302.13971. <https://arxiv.org/abs/2302.13971>

- [58] Tulving, E. (1972). Episodic and semantic memory. In *Organization of Memory*,. [https://archive.org/details/organizationofme0000unse\\_j3u4/page/n9/mode/2up](https://archive.org/details/organizationofme0000unse_j3u4/page/n9/mode/2up)
- [59] Vaswani, A., Shazeer, N., Parmar, N., et al. (2017). Attention is all you need. *NeurIPS 2017*. <https://arxiv.org/abs/1706.03762>
- [60] Vrandečić, D. and Krötzsch, M. (2014). Wikidata: A free collaborative knowledgebase. *Communications of the ACM*, 57(10):78–85. <https://doi.org/10.1145/2629489>
- [61] Walker, M. P. and Stickgold, R. (2004). Sleep-dependent learning and memory consolidation. *Neuron*, 44(1):121–133. <https://doi.org/10.1016/j.neuron.2004.08.031>
- [62] Wang, S., Yu, E., Love, O., Zhang, T., et al. (2026). MemMachine: A ground-truth-preserving memory system for personalized AI agents. arXiv:2604.04853. March 2026. <https://arxiv.org/abs/2604.04853>
- [63] Wei, J., Tay, Y., Bommasani, R., et al. (2022). Emergent abilities of large language models. *Transactions on Machine Learning Research*. <https://arxiv.org/abs/2206.07682>
- [64] Weston, J., Chopra, S., and Bordes, A. (2015). Memory networks. *ICLR 2015*. <https://arxiv.org/abs/1410.3916>
- [65] Wu, Y., Rabe, M. N., Hutchins, D., and Szegedy, C. (2022). Memorizing transformers. *ICLR 2022*. <https://arxiv.org/abs/2203.08913>
- [66] Wu, D., Wang, H., Yu, W., Zhang, Y., Chang, K., and Yu, D. (2025). Long-MemEval: Benchmarking chat assistants on long-term interactive memory. *ICLR 2025*. arXiv:2410.10813. <https://arxiv.org/abs/2410.10813>
- [67] Xu, W., Liang, Z., Mei, K., Gao, H., Tan, J., and Zhang, Y. (2025). A-MEM: Agentic memory for LLM agents. *NeurIPS 2025*. arXiv:2502.12110. <https://arxiv.org/abs/2502.12110>
- [68] Yu, E., et al. (2026). Agentic memory: Learning unified long-term and short-term memory management for LLM agents. arXiv:2601.01885. <https://arxiv.org/abs/2601.01885>
- [69] Zaheer, M., Guruganesh, G., Dubey, A., et al. (2020). Big Bird: Transformers for longer sequences. *NeurIPS 2020*. <https://arxiv.org/abs/2007.14062>

- [70] Zhang, G., et al. (2025). Memory in the age of AI agents: A survey. arXiv:2512.13564. December 2025. <https://arxiv.org/abs/2512.13564>
- [71] Zheng, L., Chiang, W.-L., Sheng, Y., et al. (2023). Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. *NeurIPS 2023*. <https://arxiv.org/abs/2306.05685>

## A MemEval-7 Sample Items

---

### A.1 D1 Episodic Recall Sample

**Turn 3 (Session 1):**

“I was born in Karachi, Pakistan, and moved to Amsterdam when I was 12 years old.”

**Probe question (Turn 47, same session):**

“Where was I born, and how old was I when I moved to Amsterdam?”

**Ground truth:** Born in Karachi, Pakistan; moved to Amsterdam at age 12.

**Scoring:** 1.0 for both facts correct; 0.5 for one; 0.0 for neither.

### A.2 D3 Cross-Session Persistence Sample

**Session 1, Turn 7:**

“I should mention that I am vegetarian and I am actively trying to lose weight.”

**Session 2 (14 days later), Turn 1:**

“Can you recommend a good dinner recipe for tonight?”

**Expected behavior:** The system should proactively recommend a vegetarian, weight-appropriate recipe without requiring the user to re-state constraints.

**Scoring:** Expert evaluation 0–2 on appropriate preference application (0=ignored; 1=partially applied; 2=correctly applied both constraints).

### A.3 D7 Forgetting Robustness Sample

**Setup:** Memory pool after 30 days of interaction, containing 1,408 entries (C2 configuration).

**Gold retain set (all three must be retained):**

1. “User’s name is Ahmed.” (importance=0.60, access\_count=12)
2. “User is allergic to shellfish.” (importance=0.95, access\_count=2)
3. “User is a PhD student in computer science at the University of Amsterdam.” (importance=0.80, access\_count=5)

**Gold evict set (all three should be evicted):**

1. “User mentioned it was raining today.” (importance=0.05, access\_count=0)
2. “User asked about bus schedules on Tuesday.” (importance=0.10, access\_count=0)
3. “User commented on a news story.” (importance=0.12, access\_count=1)

**Scoring:** Precision = fraction of system-evicted memories that are in gold evict set; Recall = fraction of gold evict set that system evicted; F1 = harmonic mean.



## B Full Hyperparameter Settings

Table 25: Complete MAC Hyperparameter Settings (All Configurations)

| Parameter                         | Value                  | Description                         |
|-----------------------------------|------------------------|-------------------------------------|
| Base LLM                          | GPT-4o                 | OpenAI API, temperature=0.1         |
| Embedding model                   | text-embedding-3-large | 3072-dim, PCA to 1536               |
| Vector store                      | ChromaDB               | HNSW index, cosine metric           |
| BM25                              | BM25Okapi              | rank-bm25 library, k1=1.5, b=0.75   |
| RRF constant $k$                  | 60                     | Standard RRF configuration          |
| Retrieval top- $k$                | 5                      | Per level per turn                  |
| Importance threshold              | 0.40                   | Minimum to store episodic memory    |
| L0 promotion threshold $\theta_0$ | 20 entries             | Episodes before working summary     |
| L1 promotion threshold $\theta_1$ | 10 entries             | Summaries before reflection         |
| Forgetting rate $\lambda$         | 0.03/day               | Temporal decay rate                 |
| $w_{\text{time}}$                 | 0.35                   | Forgetting: time weight             |
| $w_{\text{importance}}$           | 0.45                   | Forgetting: importance weight       |
| $w_{\text{frequency}}$            | 0.20                   | Forgetting: frequency weight        |
| Eviction threshold                | 0.28                   | Evict if retention score below this |
| Min. importance gate              | 0.80                   | Never evict above this importance   |
| DBSCAN $\epsilon$                 | 0.15                   | Cosine distance threshold           |
| DBSCAN min_samples                | 2                      | Minimum cluster size                |
| Forgetting daemon                 | Daily                  | Runs asynchronously every 24h       |
| Context injection budget          | 2000 tokens            | Max tokens for memory injection     |

## C System Prompt Templates

---

### C.1 Memory Writer Extraction Prompt

**Companion Notebook (§2).** The full `EXTRACT_PROMPT` template including the complete importance scale rubric (0.0 = trivial through 1.0 = critical), all seven type categories, and the example output format is defined verbatim in the `MemoryWriter` cell of `llmmemory`. The extraction prompt used in all experiments is identical to the one shown there.

### C.2 Reflection Generation Prompt

**Companion Notebook (§3).** The full `REFLECTION_PROMPT` template including the three-part generation criteria (cross-memory synthesis, personalization utility, confident specificity) and the example JSON output format is used verbatim in the `MemoryHierarchy._promote_l1_to_l3()` method of `llmmemory`.

## D Mathematical Proofs

---

### D.1 Proof of Proposition 4.1

*Proof.* Let  $N(t) = |\mathcal{M}(t)|$  denote the memory pool size at time  $t$ , and let  $\mu > 0$  be the constant write rate (new memories per unit time).

**Case 1: No forgetting ( $\lambda = 0$ ).**  $\dot{N}(t) = \mu$ , so  $N(t) = \mu t + N(0)$ . Retrieval precision satisfies:

$$P(t) = \frac{|\mathcal{M}_{\text{relevant}}(t)|}{N(t)} \leq \frac{c}{N(t)} = \frac{c}{\mu t + N(0)} \xrightarrow{t \rightarrow \infty} 0 \quad (51)$$

where  $c = |\mathcal{M}_{\text{relevant}}|$  is bounded (only finitely many memories are relevant to any query). Therefore,  $P(t) \rightarrow 0$  as  $t \rightarrow \infty$ .

**Case 2: Forgetting with rate  $\lambda > 0$ .** The pool dynamics follow:

$$\dot{N}(t) = \mu - \lambda N(t) \quad (52)$$

This is a first-order linear ODE with solution:

$$N(t) = \frac{\mu}{\lambda} + \left(N(0) - \frac{\mu}{\lambda}\right) e^{-\lambda t} \xrightarrow{t \rightarrow \infty} N^* = \frac{\mu}{\lambda} \quad (53)$$

At equilibrium:

$$P^* = \frac{c}{N^*} = \frac{c\lambda}{\mu} > 0 \quad \text{for all } \lambda > 0 \quad (54)$$

Therefore,  $P^* = c\lambda/\mu > 0 > \lim_{t \rightarrow \infty} P(t)|_{\lambda=0}$ .

For the consolidation policy,  $\lambda$  is not constant but increases with  $N$  (more clusters form at larger pool sizes), providing a stronger convergence guarantee than linear forgetting.  $\square$   $\square$